小车传感器及串口通讯

一、OLED 显示屏

1、OLED 简介

OLED (OrganicLight-Emitting Diode) 又称为有机电激光显示、有机发光半导体（OrganicElectroluminesence Display，OLED）。OLED 属于一种电流型的有机发光器件，是通过载流子的注入和复合而致发光的现象，发光强度与注入的电流成正比。OLED 在电场的作用下，阳极产生的空穴和阴极产生的电子就会发生移动，分别向空穴传输层和电子传输层注入，迁移到发光层。当二者在发光层相遇时，产生能量激子，从而激发发光分子最终产生可见光。一般而言，OLED 可按发光材料分为两种：小分子 OLED 和高分子 OLED（也可称为 PLED）。OLED 是一种利用多层有机薄膜结构产生电致发光的器件，它很容易制作，而且只需要低的驱动电压，这些主要的特征使得 OLED 在满足平面显示器的应用上显得非常突出。OLED 显示屏比 LCD 更轻薄、亮度高、功耗低、响应快、清晰度高、柔性好、发光效率高，能满足消费者对显示技术的新需求。全球越来越多的显示器厂家纷纷投入研发，大大的推动了 OLED 的产业化进程。

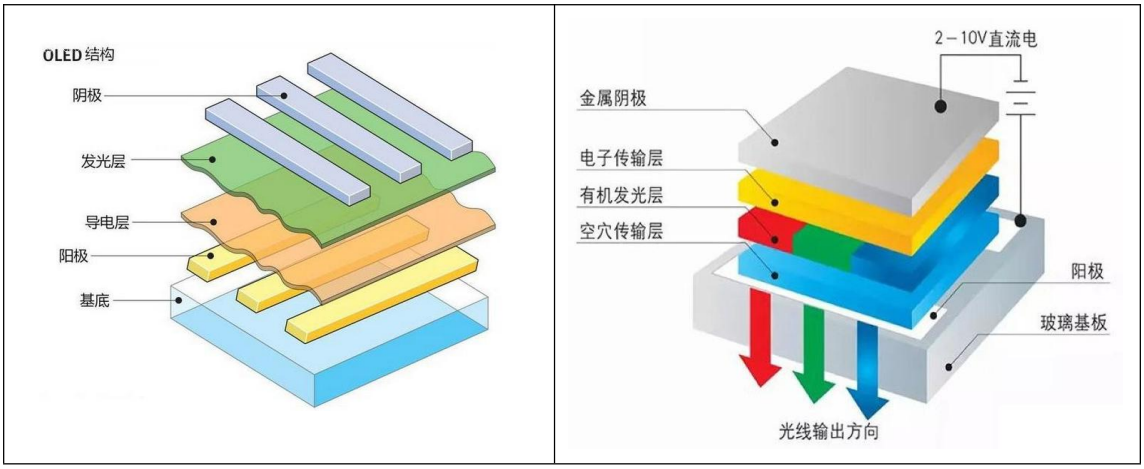


图 1.1.1 OLED 的结构及发光原理

2、OLED 模块

本实验使用的模块实物如图 1.2.1 所示。此模块采用的是 4 线 SPI 接口，还有一种模块是采用的 2 线 I2C 接口。

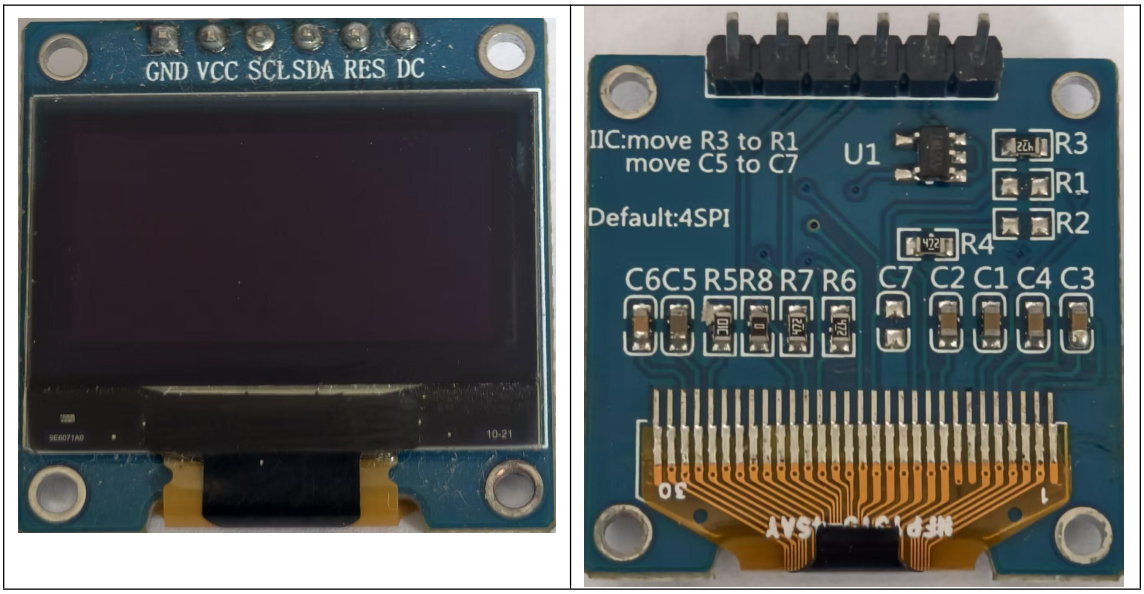


图 1.2.1 OLED 模块正反面

模块主要参数：

①、高分辨率:128*64；

②、广可视角度:大于 160°；

③、低功耗:正常显示时 0.04W;

④、宽供电范围:直流 3.3V-5V;

⑤、工业级:工作温度范围-30°C~70°C;

⑥、体积:29.2mm*27.5mm;

⑦、通信方式:SPl;

⑧、亮度、对比度可以通过程序指令控制。

⑨、OLED 屏幕内部驱动芯片:SSD1306。

引脚定义:

①、DC:数据或命令切换。

②、RES:复位。

③、SDA:双向数据线。

④、SCL:时钟信号线。

⑤、VCC:电源正 3.3V。

⑥、GND:电源负。

3、OLED 屏的坐标及显示原理框图

在 OLED 坐标系中，左上角是原点，向右是 X 轴，向下是 Y 轴。如图 1.3.1 所示。



图 1.3.1 OLED 屏幕像素坐标

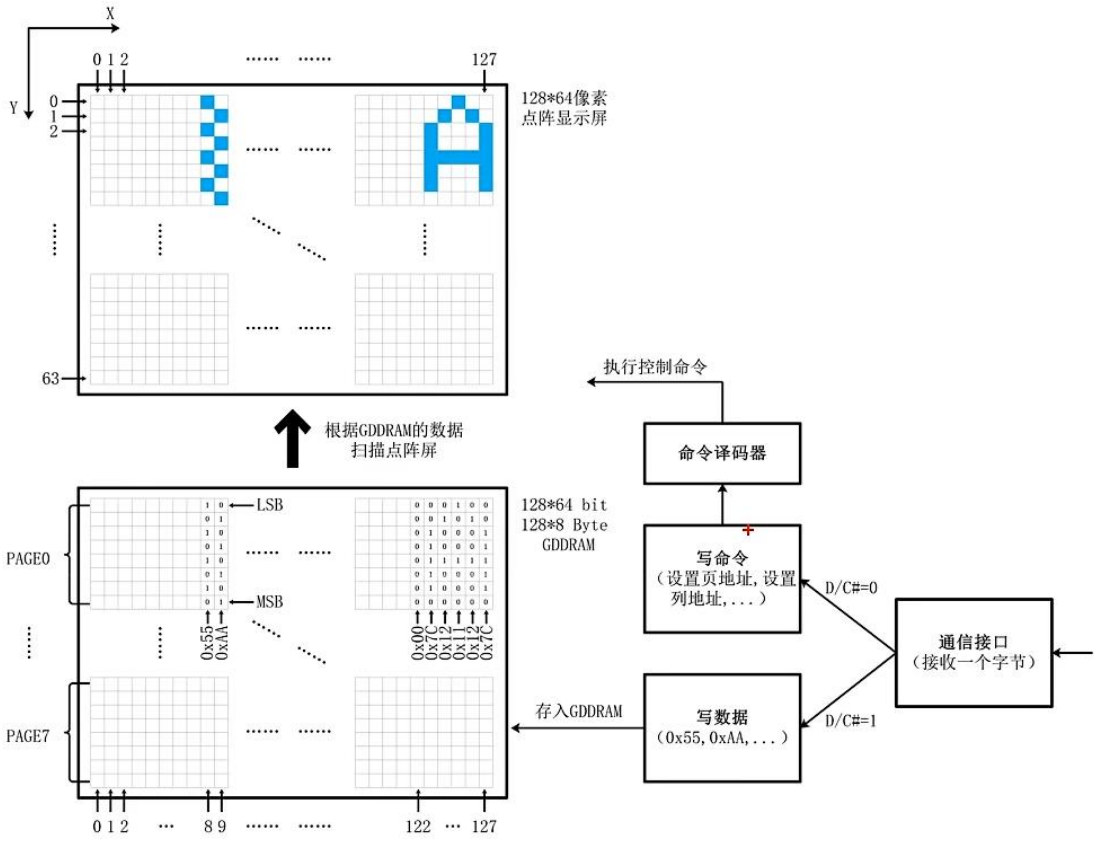


图 1.3.2 OLED 屏幕显示原理框图

4、OLED 模块编程应用

C50C 控制板的 OLED 接口如图 1.4.1 所示，其使用的 CPU 是 STM32F407VET6。C10B 控制板的 OLED 接口如图 1.4.2 所示，其使用的 CPU 是 STM32F103RCT6。

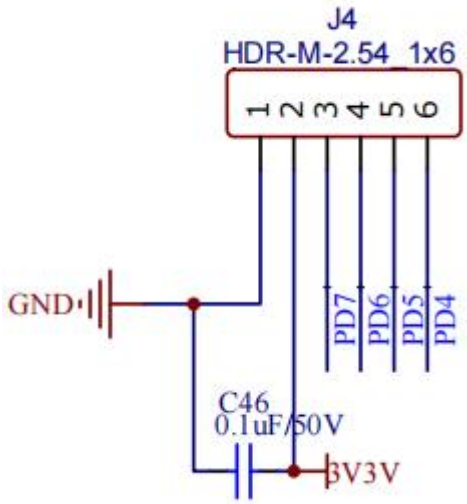


图 1.4.1 C50C OLED 模块接口

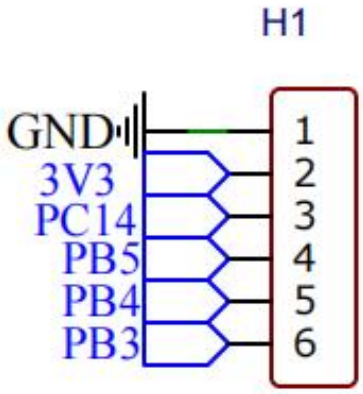


图 1.4.2 C10B OLED 模块接口

4.1 通讯接口引脚定义

C50C 控制板编程时定义的端口及引脚如下：

```
#define OLED_SCL_PORT     GPIOD //port
#define OLED_SDA_PORT     GPIOD
#define OLED_RES_PORT     GPIOD
#define OLED_DC_PORT      GPIOD

#define OLED_SCL_PIN      GPIO_Pin_7 //pin
#define OLED_SDA_PIN      GPIO_Pin_6
#define OLED_RES_PIN      GPIO_Pin_5
#define OLED_DC_PIN       GPIO_Pin_4
```

C10B 控制板编程时定义的端口及引脚如下：

```
#define OLED_DC_Pin         GPIO_PIN_3
#define OLED_DC_GPIO_Port   GPIOB
#define OLED_RES_Pin        GPIO_PIN_4
#define OLED_RES_GPIO_Port  GPIOB
#define OLED_SDA_Pin        GPIO_PIN_5
#define OLED_SDA_GPIO_Port  GPIOB
#define OLED_SCL_Pin        GPIO_PIN_14
#define OLED_SCL_GPIO_Port  GPIOC
```

4.2 OLED 屏写时序

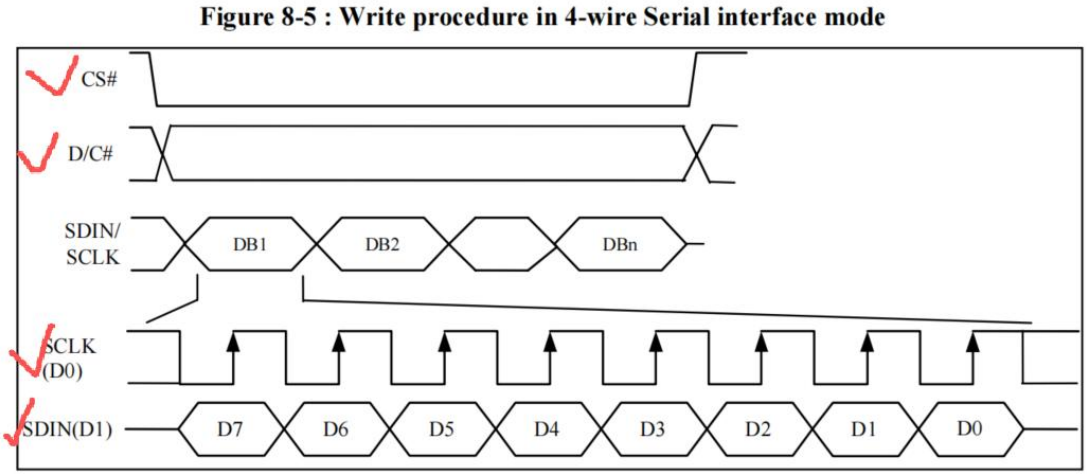


图 1.4.3 OLED 写字节的时序

OLED 的原子操作（bsp_oled.c）：

```
/**********************************************************************
函数功能：向 OLED 写入一个字节
入口参数：dat:要写入的数据/命令，cmd:数据/命令标志 0,表示命令;1,表示数据
返回  值：无
说    明：参照手册 P17-18
**********************************************************************/
void OLED_WR_Byte(uint8_t dat,uint8_t cmd)
{
    uint8_t i;
```

```
    if(cmd)
    {
        OLED_DC_Set();    //写数据
    }

    else
    {
        OLED_DC_Clr();     //写命令
    }

    for(i=0;i<8;i++)
    {
        OLED_SCLK_Clr();
        if(dat&0x80)
        {
             OLED_SDIN_Set();
        }

        else
        {
             OLED_SDIN_Clr();
        }

        OLED_SCLK_Set();  //时钟上升沿把数据移进 8-bit 的移位寄存器
        dat<<=1;
    }
    OLED_DC_Set();
}
```

4.3 OLED 正常操作流程

整体两大阶段

1）初始化流程（上电仅执行 1 次）

2）根据需要绘图 + 刷新显示流程（循环执行）

上电初始化完整流程：

1）上电复位

2）发送指令 `0xAE`：关闭屏幕显示

3）时钟配置 `0xD5` + 参数 0x80：振荡频率 + 分频

4）复用行数 `0xA8` + 参数 0x3F：设置 64 行 COM

5）垂直偏移 `0xD3` + 参数 0x00：无画面上下偏移

6）显示起始行 `0x40`：RAM 从第 0 行开始显示

7）COM 硬件配置 `0xDA` + 0x12：适配 128*64 屏引脚排布

8）预充电周期 `0xD9` + 0xF1

9）VCOMH 消隐电压 `0xDB` + 0x30

10）电荷泵 `0x8D` + 0x14：开启内部升压（屏幕点亮必备）

11）显存寻址模式设置（默认页寻址）

12）发送指令 `0xAF`：打开屏幕显示

13）调用清屏函数：清空内部缓存 GRAM + 刷新到 OLED 硬件显存

实际绘图+屏幕刷新显示流程：

1）根据业务需要，调用绘图函数：DrawPoint/DrawLine/ShowString 在指定位置绘图（此步不是真正在屏幕上显示内容，只是修改单片机内部 GRAM 数组）

2）调用 OLED_Refresh()函数刷新屏幕内容（真正操作硬件）

4.4 C10B 工程 OLED 相关程序解读

OLED 程序要求会使用即可，能在（x,y）处显示字符或者数字，作为人机交互的一种手段。OLED 不带字库，例程里只提供了常用 ASCII 表的 2 种点阵字体，有些特殊符号、图形或者汉字需要自己取模制作，具体使用方法可以参考程序中的注释（oledfont.h）。

二、超声波传感器 SR04

1、认识模块

HC-SR04 是一款低成本、易上手的超声波测距模块，实物如图 2.1.1 所示。



图 2.1.1 HC-SR04 超声波测距传感器模块

主要技术参数

①：使用电压：DC---5V

②：静态电流：小于 2mA

③：电平输出：高 5V

④：电平输出：底 0V

⑤：感应角度：不大于 15 度

⑥：探测距离：2cm-400cm （2 cm 以内是盲区，数据不可信）

⑦： 高精度 约+-3mm

⑧： 接线方式：VCC、trig（控制端）、 echo（接收端）、 GND

主要应用场景有避障、液位检测等。

2、什么是超声波

超声波是一种频率高于 20000 赫兹的声波，它的方向性好，穿透能力强，易于获得较集中的声能，在水中传播距离远，可用于测距、测速、清洗、焊接、碎石、杀菌消毒等。在医学、军事、工业、农业上有很多的应用。超声波因其频率下限大于人的听觉上限而得名。科学家们将每秒钟振动的次数称为声音的频率，它的单位是赫兹(Hz)。我们人类耳朵能听到的声波频率为 20Hz-20000Hz。因此，我们把频率高于 20000 赫兹的声波称为“超声波”。

3、超声波探头

是利用超声波的特性而研制的传感器，超声波传感器的典型结构如图 2.3.1 所示。它是把成正方形的两个压电晶片(亦称双晶振子)按照相反的极性粘贴在一起，再引出两个电极。压电晶片上面有金属震动板和圆锥形振子。圆锥形振子具有很强的方向性，便于发送和接收超声波。超声波传感器采用金属或塑料外壳，其顶部有屏蔽栅。

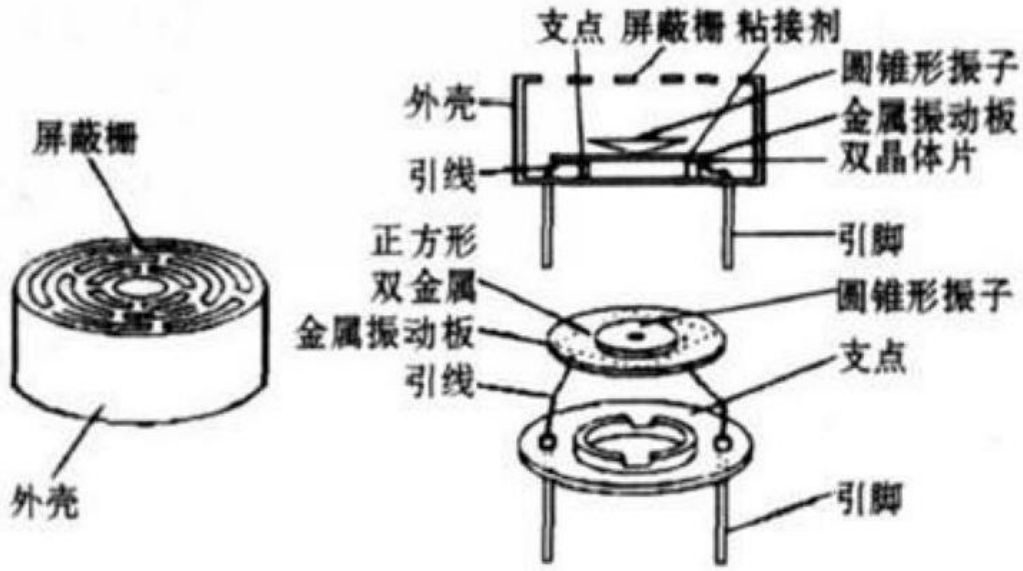


图 2.3.1 超声波传感器的典型结构

4、超声波测距原理

超声波具有频率较高，沿直线传播、方向性好、绕射小、穿透力强、传播速度慢(约 340m/s,与声速相同)等特点。超声波对固体和液体的穿透能力强，尤其对于在阳光下不透明的固体，可以穿透几十 m 的深度。超声波遇到杂质或分界面时会产生反射波，利用这一特性可构成超声波探伤仪或测距仪。超声波遇到移动物体时会产生多普勒效应(DopplerEffect)，使接收到的频率发生变化，由此可制成多普勒测距系统。超声波测距原理是超声波发射探头发出的超声波脉冲，经媒质(空气)传到物体表面，反射后通过媒质(空气)传到接收探头，测出超声脉冲从发射到接收所需的时间，根据媒质中的声速，求得从探头到物体表面之间的距离。设探头到物体表面的距离为 L，超声在空气中的传播速为 v，从发射到接收所需的传播时间为 t，则有：L=vt

/2。由此可见，被测距离 L 与传播时间之间具有确定的函数关系，只要能测出时间 t，即可求出距离 L，通过软件实现直接在显示器上显示 L 的值。

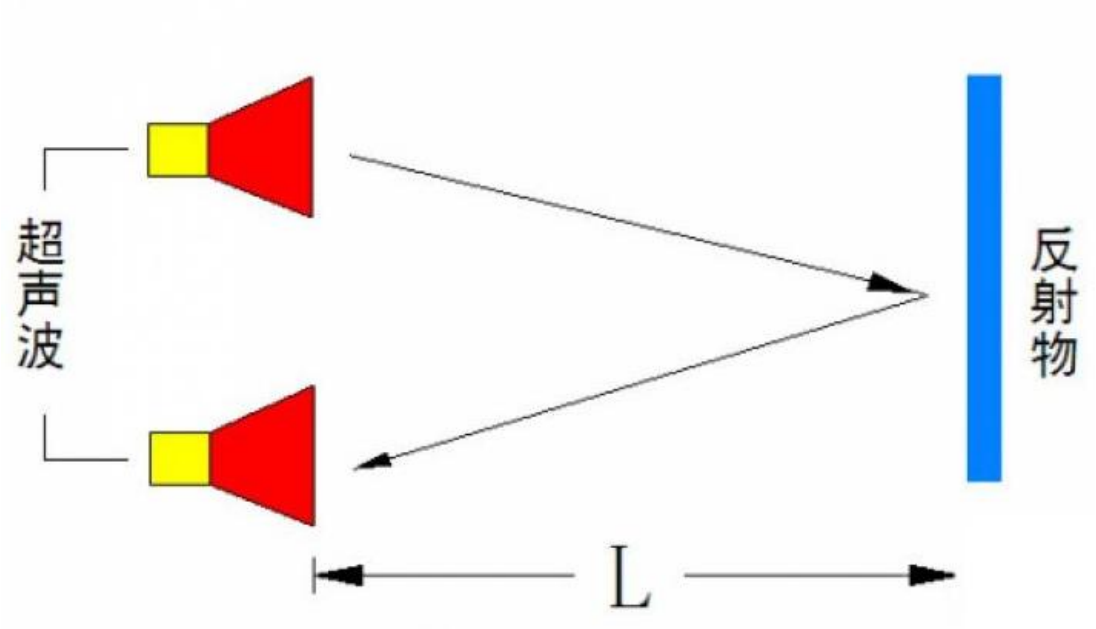


图 2.4.1 超声波测距原理示意图

5、测距实现

超声波测距的时序图如图 2.5.1 所示，实现步骤如下：

①设置引脚：将 HC-SR04 模块的 Trig 引脚连接到单片机的一个 GPIO 输出引脚，将 Echo 引脚连接到单片机的一个 GPIO 输入引脚。

②发射信号：通过将 Trig 引脚设置为高电平，持续不小于 10 微秒（建议放到 15us），然后再将其拉低，来触发模块发射超声波信号。

③接收信号：开始计时，等待 Echo 引脚由低电平变为高电平，

记录时间。

④计算距离：根据时间差和声速的关系，计算出测量到的距离。

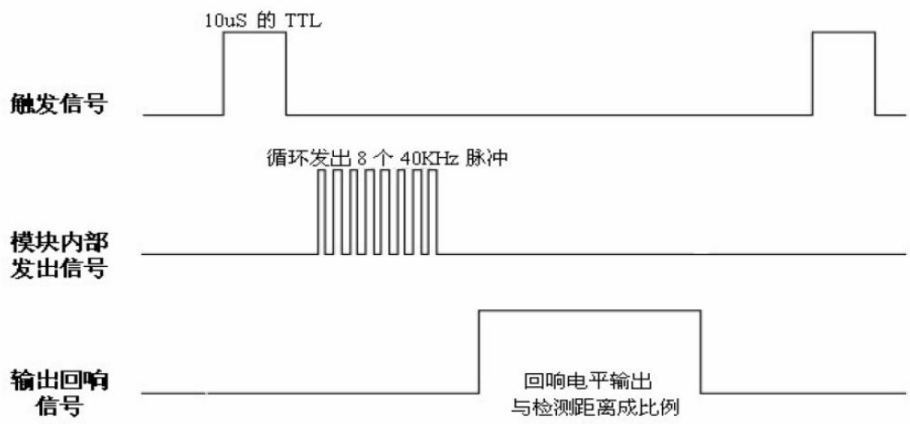


图 2.5.1 超声波测距原理时序图

两次测距间隔建议≥60ms，防止回波干扰。

超声波测距的实现方式有表 2.5.1 所示的几种方式。

表 2.5.1 超声波测距的实现方式

实现方式	实时性	测量精度	CPU 占用	外设需求	推荐使用场景
软件 GPIO 轮询	极差（阻塞）	一般(依赖 DWT)	极高	仅普通 GPIO	教学入门、简单演示
外部中断 + 定时器计数	中等	高	中高（频繁中断）	GPIO + 独立定时器	单路超声波、简易小车
单通道定时器输入捕获	优秀	极高	低	1 路定时器捕获通道	绝大多数正式工程（首选）
门控 / 双通道输入捕获	最优	最高	极低	定时器双通道 / 门控资源	多路同步采集、低功耗高精度设备

6、编程应用

C10B 控制板上超声波接口如图 2.6.1 所示。实验中采用**单通道定时器输入捕获方式**来计算超声波的传播时间。

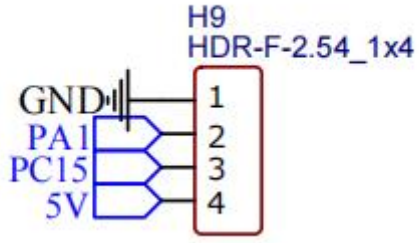


图 2.6.1 超声波传感器硬件接口图

注意 PA1 引脚原理图上超声波输出的 echo 引脚是直接连接到 MCU 的，因为 echo 信号是 5V 电平，而单片机是 3.3V 供电，稳妥的做法是经过电阻分压再连接。这里为什么能直接连接呢？因为 STM32 单片机部分引脚标注为 FT 引脚，当 GPIO 用时可以容忍 5v 输入。

测 Echo 高电平持续了多久为什么采用“输入捕获”呢?

方案	做法	评价
外部中断 + 定时器	EXTI 上升沿开定时器，下降沿读值	能用；但受**中断响应延迟**影响，精度差几 μs
输入捕获	边沿到来瞬间，硬件自动把 CNT 锁存进 CCR	**硬件锁存，精度 = 1 个计数周期**，与中断响应快慢无关

输入捕获就是"边沿一到，芯片自己按下快门，把当前计数值拍进 CCR 寄存器，然后才去通知 CPU"。CPU 什么时候来取，不影响照

片里拍到的数字——这就是它比外部中断准的原因。

那为什么采用单通道不采用双通道呢？

方式	接法	特点
双通道	Echo 同时接 CH1 和 CH2，CH1 抓上升、CH2 抓下降	逻辑简单，但占两个通道、多一根线
单通道	Echo 只接 CH1，一轮测量内**动态切换捕获极性**	省资源；**难点在状态机与极性切换**，更能训练思维

单通道的核心动作只有两步：上升沿抓到 → 立刻把极性改成下降沿；下降沿抓到 → 算完改回上升沿。用一个 sm 状态位记住"我现在在等哪个沿"。

定时器配置关键：

配置项	取值	为什么
定时器时钟	72 MHz	TIM2 在 APB1，PCLK1=36 MHz，**APB1 预分频≠1 时定时器时钟 ×2**
预分频 PSC	71	72MHz ÷ (71+1) = **1 MHz** → **1 个计数 = 1 μs**，换算最省事
重装载 ARR	0xFFFF	满量程 65535 μs ≈ 65.5 ms
计数模式	向上计数	从 0 累加到 ARR 再溢出归零

捕获极性	初始上升沿	先抓 Echo 变高那一刻
开启中断	CC2 + Update	Update（溢出）中断用来累计溢出次数

为什么必须开溢出中断？4 m 对应 Echo 高电平约 23.5 ms = 23500 个计数，而 16 位计数器最大只到 65535——看起来够？但如果测更远或计数起点没清零，就会溢出。更关键的是溢出不处理会得到完全错误的结果。所以：ovf++，最终脉宽 = CCR 值 + ovf × 65536。

超声波测距流程如图 2.621 所示。



图 2.6.2 超声波测距流程图

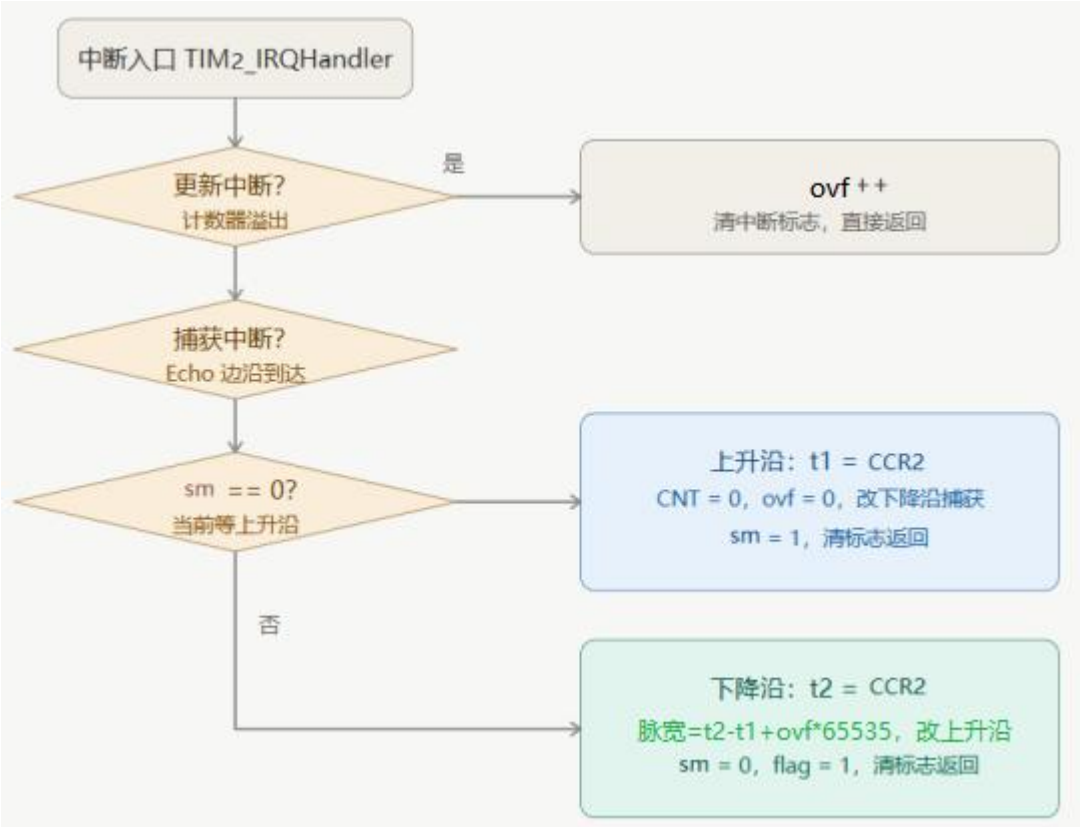


图 2.6.3 超声波测距中断程序流程图

程序结构说明：

1）主程序调用函数 MX_TIM2_Init();对定时器 2 初始化，实现在文件 tim.c 中有详细注释。

2） HAL_TIM_IC_Start_IT(&htim2,TIM_CHANNEL_2); //开启捕获中断

3）最后在 HAL_TIM_IC_CaptureCallback（）中断回调里根据 sm 状态机读取计数值、计算距离、切换上下沿捕获。

超声波的启动借助于 MPU6050 的外部中断每 50ms 启动一次。

三、蜂鸣器控制

蜂鸣器分为有源蜂鸣器和无源蜂鸣器两种，有源和无源从外观上不易辨别，可以测量其阻值。无源两引脚间阻值大约 15 欧姆，有源的话两引脚间阻值特别大，MΩ以上。C10B 和 C50C 控制板使用的都是 5V 有源蜂鸣器，其控制原理图如图 3.1。C10B 板子上的实物图如图 3.2。

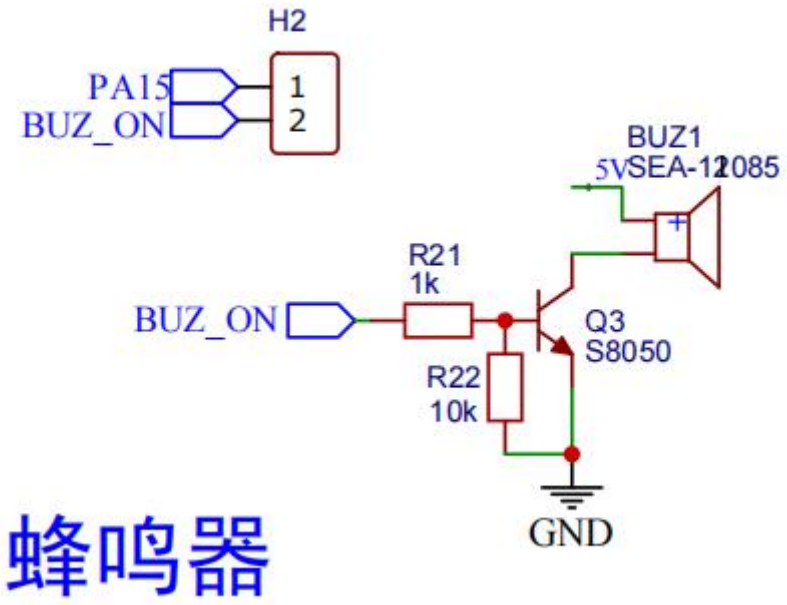


图 3.1 蜂鸣器控制原理图

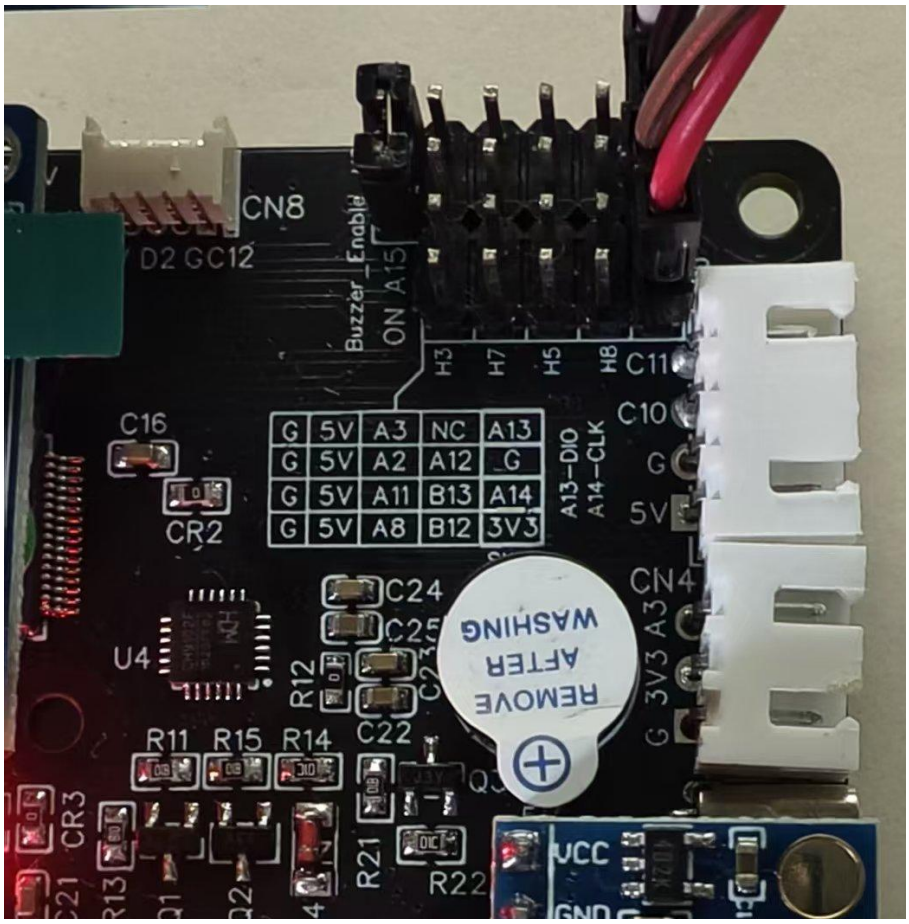


图 3.2 蜂鸣器实物图

有源蜂鸣器的编程控制比较简单，根据图 3.1，只要控制 PA15 为高电平即可让蜂鸣器发声（注意 H2 的短路跳线帽要插上）。

```
GPIO_InitStruct.Pin = beep_Pin;                    //引脚号
GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;        //推挽输出
GPIO_InitStruct.Pull = GPIO_NOPULL;                //不上拉、不下拉，浮空输出
GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;       //低速模式，功耗更低
HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);

HAL_GPIO_WritePin(GPIOA, beep_Pin, GPIO_PIN_SET);   //启动蜂鸣器
```

四、舵机控制

1、 什么是舵机：

舵机最早出现在航模运动中，用来控制飞行器的飞行姿态，比如控制飞机上升下降，前后，左右转向，以及左右侧飞，这些操控都是由机上的舵机通过连杆等传动元件带动舵面的转动来实现的。舵机因此得名：控制舵面的伺服电机。



图 4.1.1 舵机实物图

舵机是一种位置（角度）伺服的驱动器，适用于那些需要角度不断变化并可以保持的控制系统。目前在高档遥控玩具，如航模，包括飞机模型，潜艇模型，机器人，机械臂中已经使用得比较普遍。舵机是一种俗称，其实是一种伺服马达，其内部结构如图 4.1.2。 伺服马达内部包括了一个小型直流马达、一组变速齿轮、一个反馈可调电位

器及一块控制板。

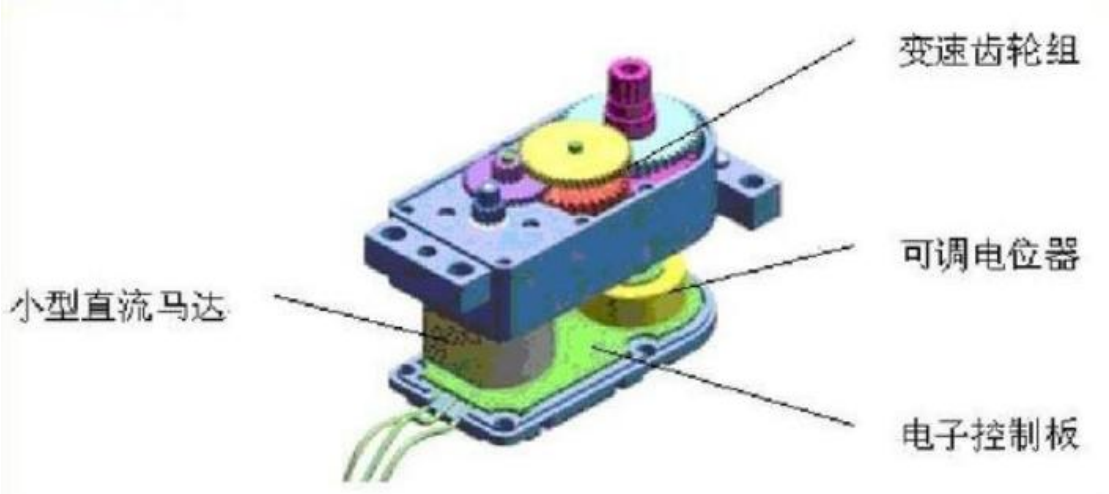


图 4.1.2 舵机内部结构

在使用中，舵机的配件通常包含一个能把舵机固定到基座上的支架以及可以套在驱动轴上的舵盘，通过舵盘上的孔可以连接其它物体构成传动模型。小舵机自带的 3 线接口可以很方便地与主板相连。

舵机的电机有有刷和无刷之分，齿轮有塑料和金属之分，输出轴有滑动和滚动之分，壳体有塑料和铝合金之分，速度有快速和慢速之分，体积有大中小三种之分等等，组合不同，价格也千差万别。需要根据应用场合选用不同类型的舵机。

2、其工作原理

控制电路板接受来自信号线的控制信号，控制电机转动，电机带动一系列齿轮组，减速后传动至输出舵盘。舵机的输出轴和位置反馈电位计是相连的，舵盘转动的同时，带动位置反馈电位计，电位计将输出一个电压信号到控制电路板，进行反馈，然后控制电路板根据所在位置决定电机转动的方向和速度，从而达到目标停止。其工作流程为：控制信号→控制电路板→电机转动→齿轮组减速→舵盘转动→位

置反馈电位计→控制电路板反馈。这样的直流电机控制方式叫闭环控制，如图 4.2.1。

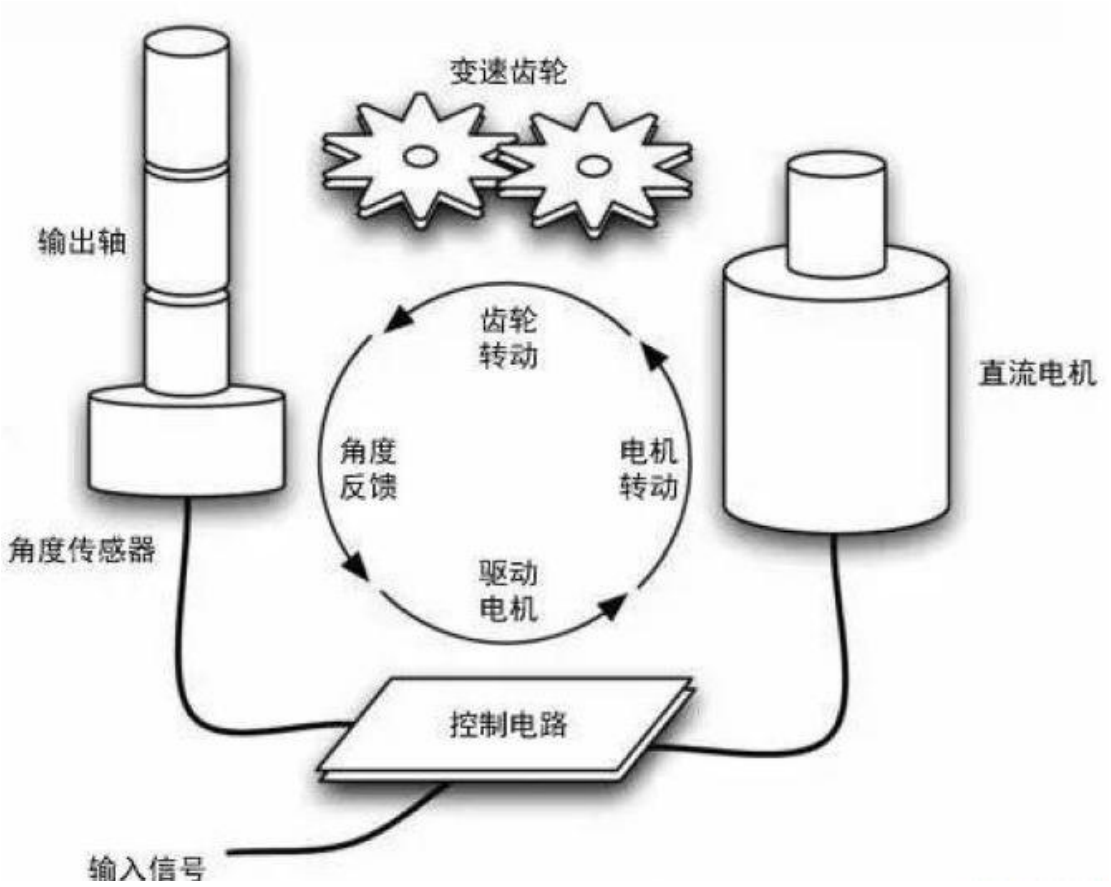


图 4.2.1 舵机内部的闭环控制

3、 舵机的控制

舵机的控制一般需要一个周期为 **20ms 左右的时基脉冲**，该脉冲的高电平部分一般为 0.5ms~2.5ms。以 180 度的伺服电机为例，对应的控制如图 4.3.1。

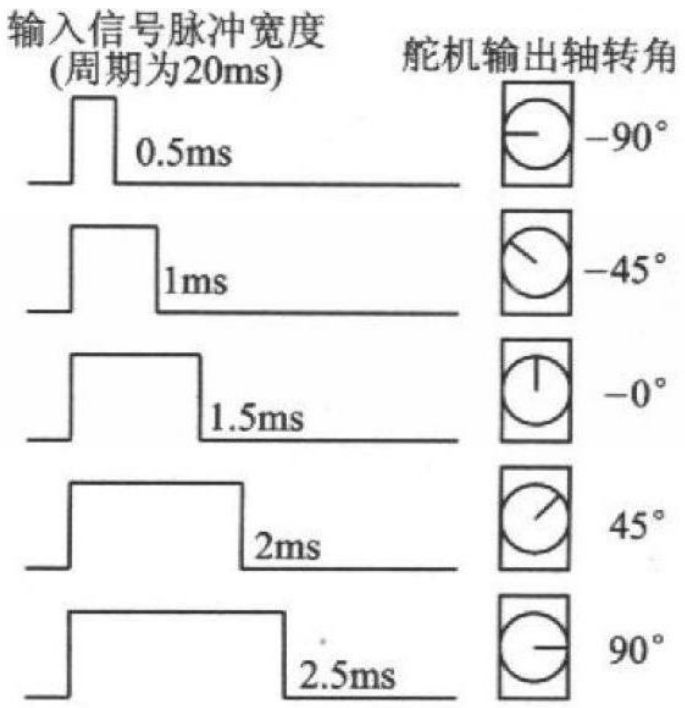


图 4.3.1 舵机输出转角与输入脉冲宽度之间的关系

也就是说，给舵机提供一定的脉宽，他的输出轴就会保持一定的对应角度，无论外接转矩怎么改变，直到给它提供另外一个脉冲信号，它才会改变输出角度到新的对应的位置上。舵机内部有一个基准电路，产生周期为 20ms，宽度为 1.5ms 的脉冲信号，有一个比较器，将外加信号与基准信号相比较，判断出方向和大小，从而产生电机的转动信号。

4、应用接线

舵机的接线有三根，如图 4.4.1。分别是黑线（地线）、红线（电源线）、黄线（信号线）。

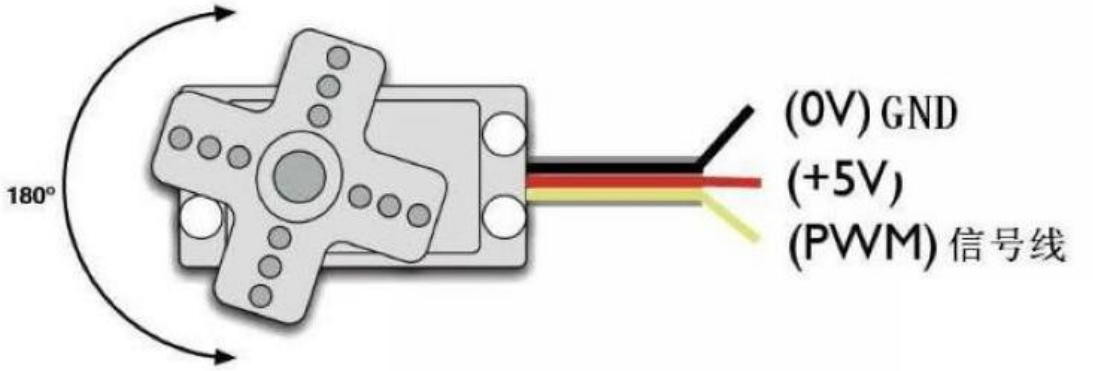


图 4.4.1 舵机接线

C10B 控制板上舵机接口原理图如图 4.4.2，实物接线如图 4.4.3。舵机插头插的时候注意方向，不要插反。

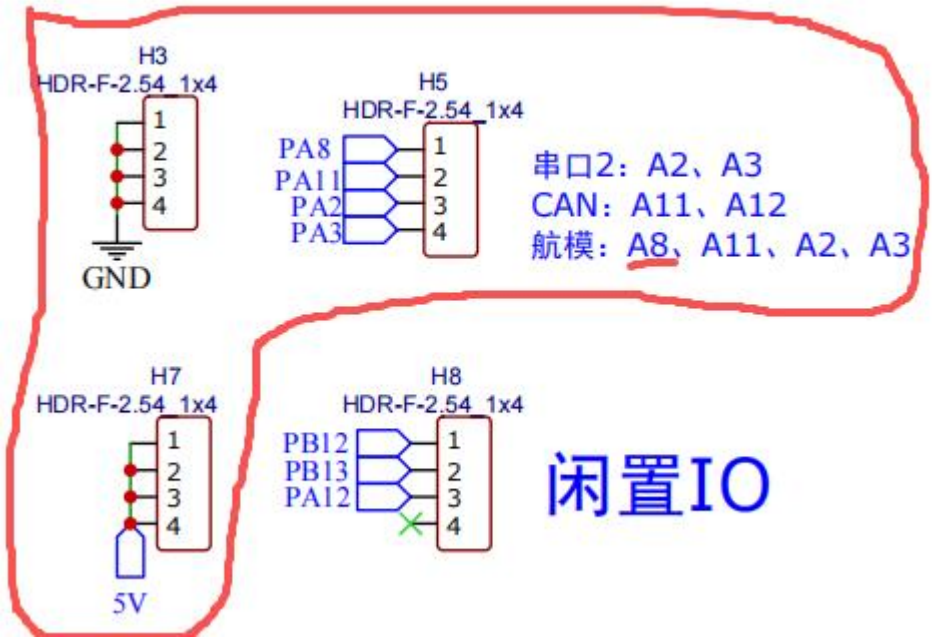


图 4.4.2 C10B 舵机接线原理图

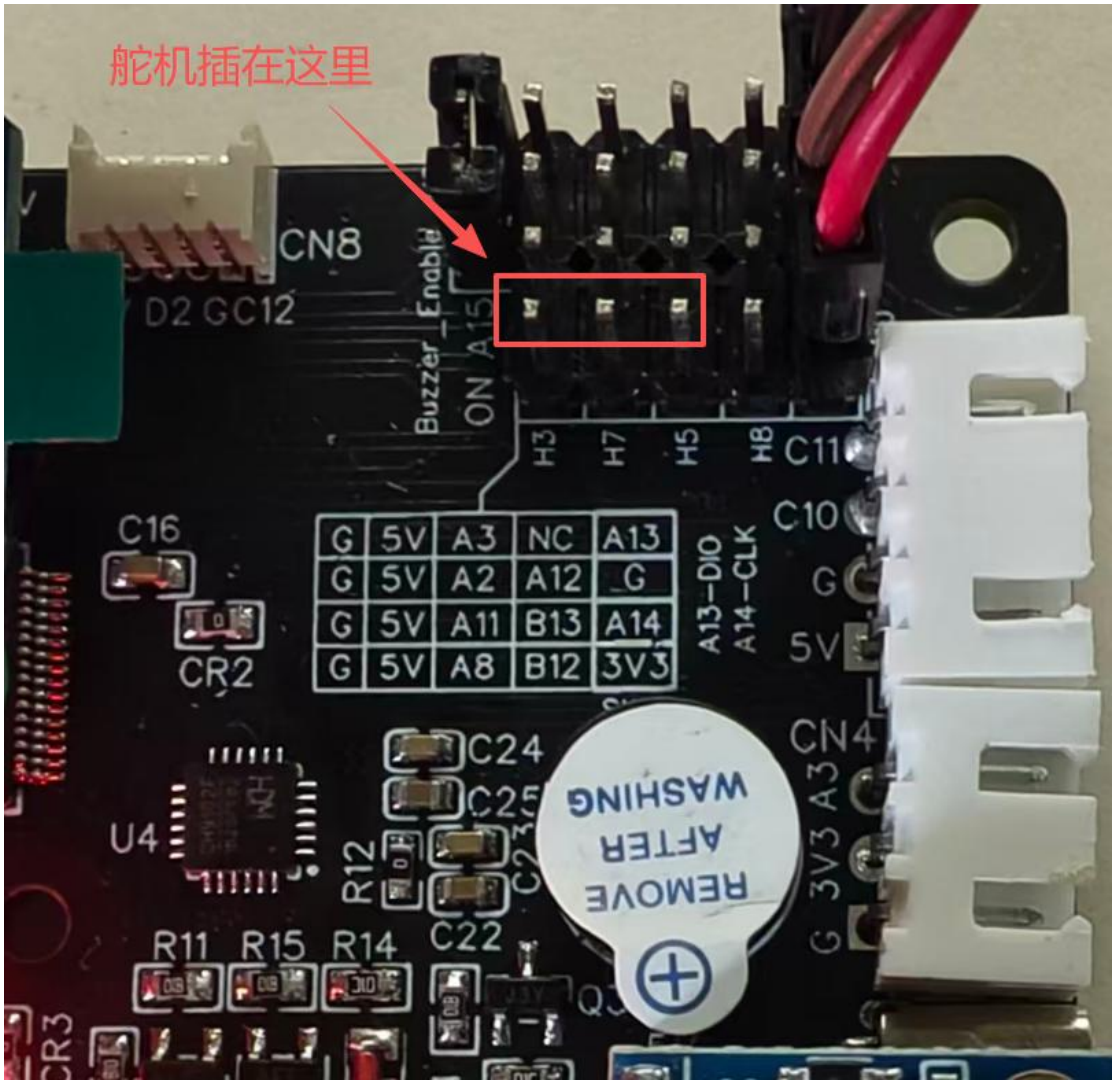


图 4.4.3 C10B 舵机接线实物图

5、编程实现

舵机控制程序流程图如图 4.5.1。程序采用非阻塞往复扫角，user 按键控制舵机的启停。

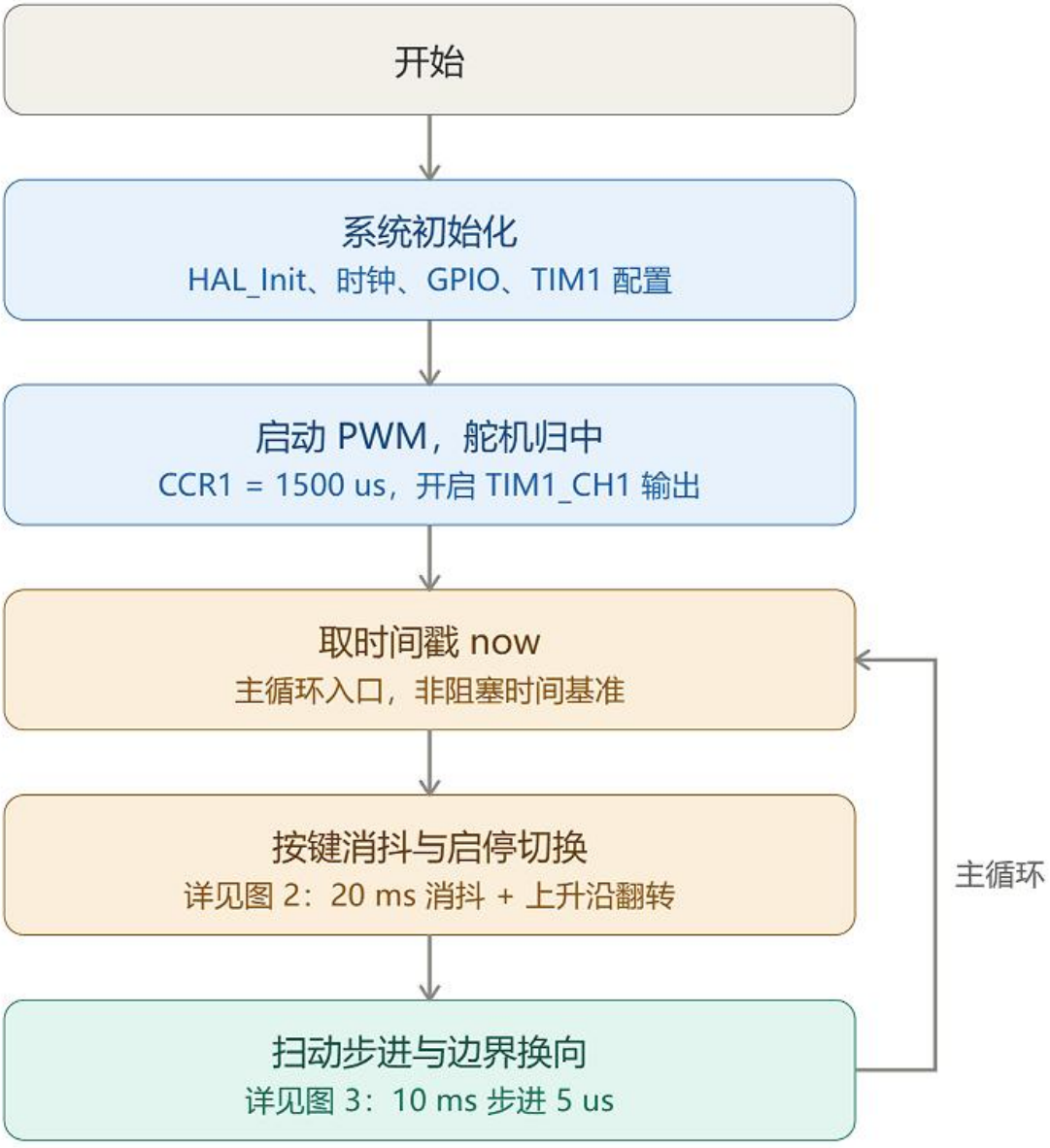


图 4.5.1 C10B 舵机控制流程图

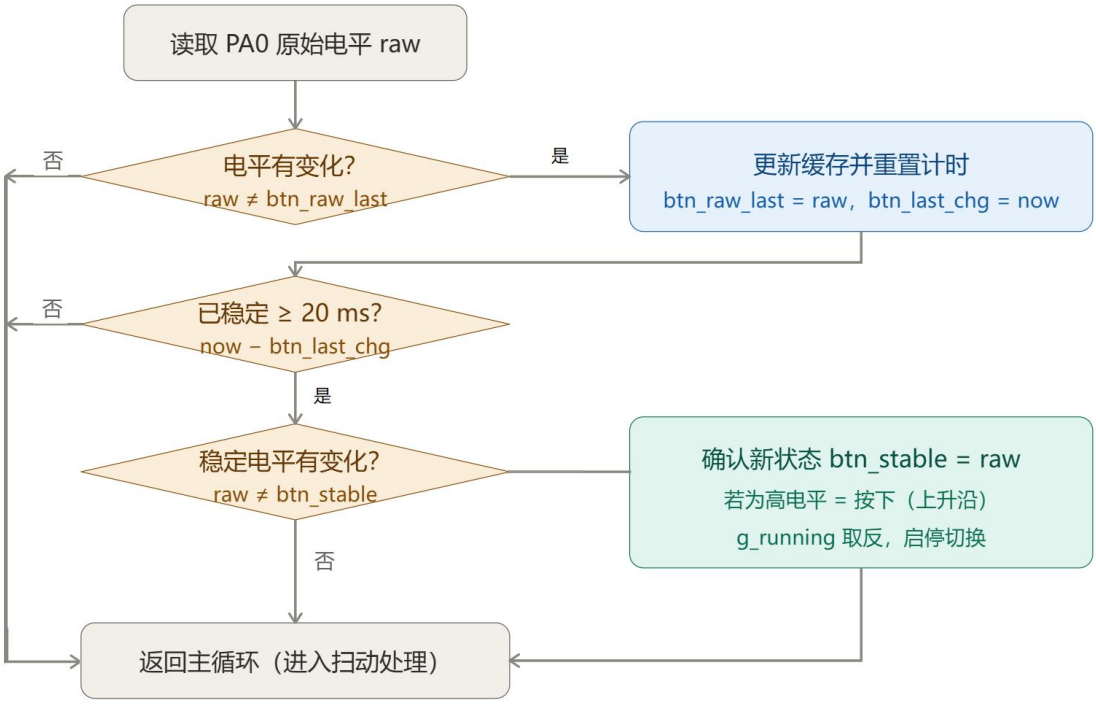


图 4.5.2 C10B 按键处理流程图

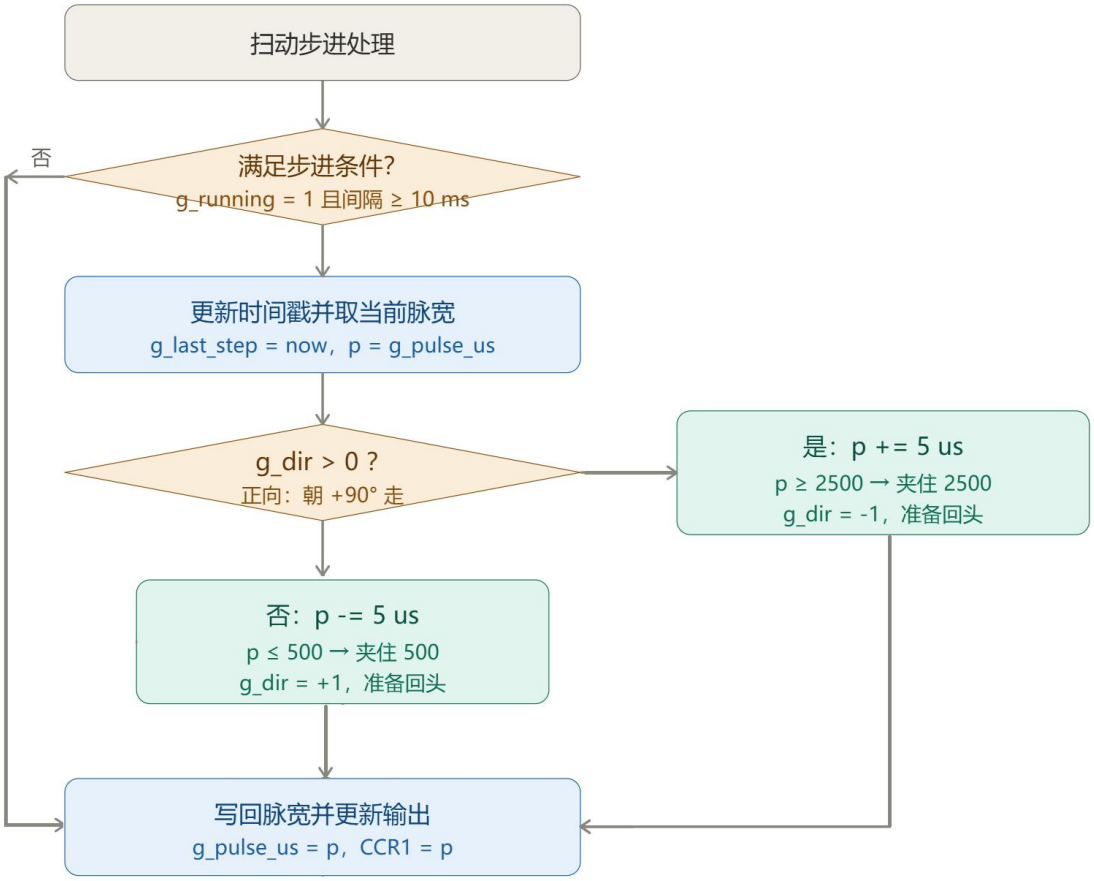


图 4.5.3 C10B 舵机步进与换向流程图

五、MPU6050 传感器

C10B 控制板 MPU6050 传感器的接口如图 5.0.1 所示。

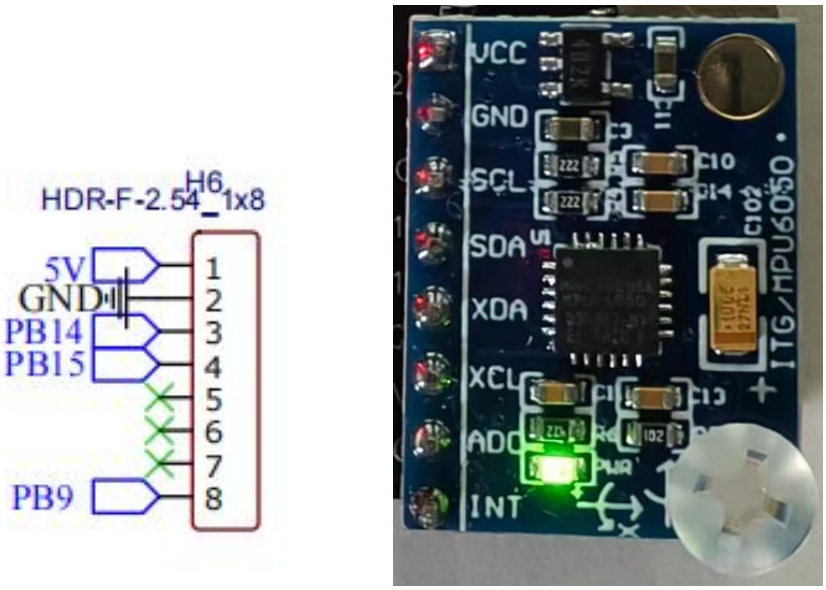


图 5.0.1 C10B 6 轴传感器连接原理图

C50C 控制板采用的是 9 轴 ICM20948 传感器，其接口如图 5.0.2 所示。

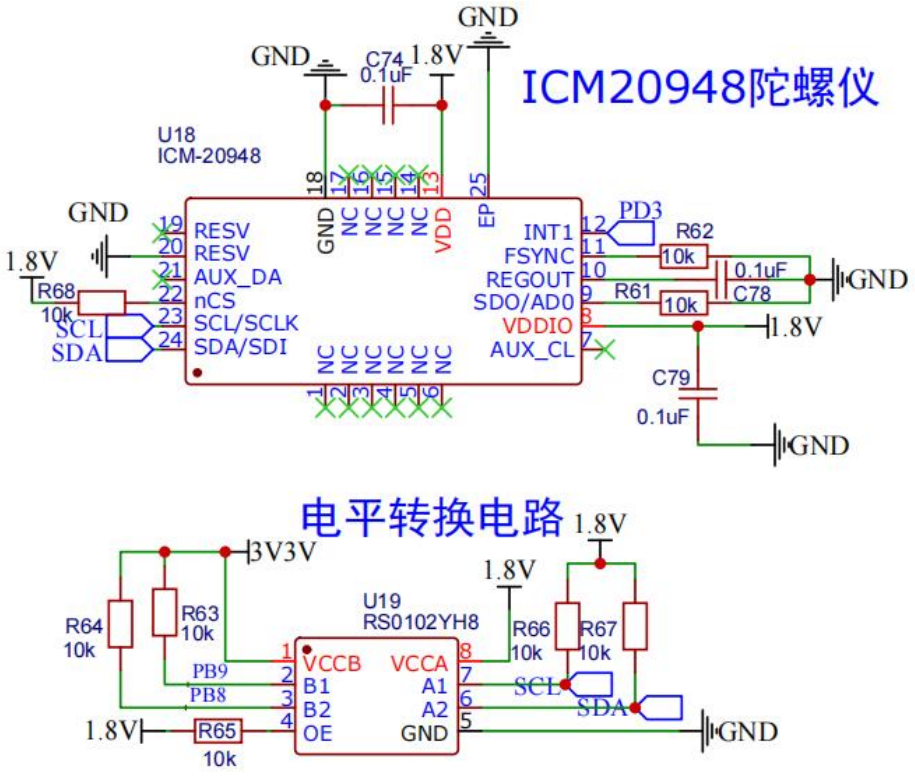


图 5.0.2 C50C 9 轴传感器连接原理图

MPU6050 和 ICM20948 都是由美国 InvenSense（应美盛）设计，2017 年日本 TDK 集团全资收购 InvenSense，二者区别如表 5.0.1 所示。

表 5.0.1 MPU6050 和 ICM20948 的区别

对比项	MPU6050	ICM20948
轴数	6 轴（加速度 + 陀螺）	9 轴（加速度 + 陀螺 + 地磁）
航向 Yaw	长期漂移，需外挂磁力计	内置地磁，无漂移
供电 IO	3.3V/5V 兼容，易接线	内核 / IO 1.8V，需电平转换
通信	仅 I2C	I2C+7MHz 高速 SPI
功耗	较高	超低功耗，适合电池设备
DMP 能力	仅 6 轴融合	硬件 9 轴融合，运动识别功能丰富
开发难度	极低，资料丰富	偏高，寄存器复杂
典型用途	平衡车、入门毕设、短期姿态	无人机、导航、穿戴、高精度姿态

InvenSense 三代产品的核心差异如表 5.0.2 所示。

表 5.0.2　InvenSense 三代产品的差异

型号	代次	核心特征
MPU6050	第一代 6 轴	仅 I2C，无地磁，DMP 仅 6 轴融合，Yaw 漂移
MPU9250	第二代 9 轴	MPU6050 陀螺内核 + 内置 AK8963,支持 I2C/SPI,DMP 仍为 6 轴，地磁需软件融合
ICM20948	第三代 9 轴	全新 MEMS 工艺，内置 AKO9916，1.8V 电平，硬件原生 9 轴 DMP，功耗大幅降低

1、MPU6050 是什么

MPU6050 是 InvenSense 公司推出的全球首款整合性 6 轴运动处理组件，相较于多组件方案，免除了组合陀螺仪与加速器时轴间差的问题，减少了安装空间。

MPU6050 内部整合了 3 轴陀螺仪和 3 轴加速度传感器，并且含有一个第二 IIC 接口，可用于连接外部磁力传感器，并利用自带的数字运动处理器（DMP: Digital Motion Processor）硬件加速引擎，通过主 IIC 接口，向应用端输出完整的 9 轴融合演算数据。有了 DMP，我们可以使用 InvenSense 公司提供的运动处理资料库，非常方便的实现姿态解算，降低了运动处理运算对操作系统的负荷，同时大大降低了开发难度。

本次实验使用的是焊接好的 MPU-6050 模块，如图 5.1.1 所示，模块对外提供 I2C 通讯接口。



图 5.1.1 MPU-6050 模块

MPU6050 内部结构与机体坐标系如图 5.1.2，MPU6050 内部框图如图 5.1.3。为帮助大家理解三个姿态角，可参见图 5.1.4。



图 5.1.2 MPU-6050 内部结构与机体坐标系

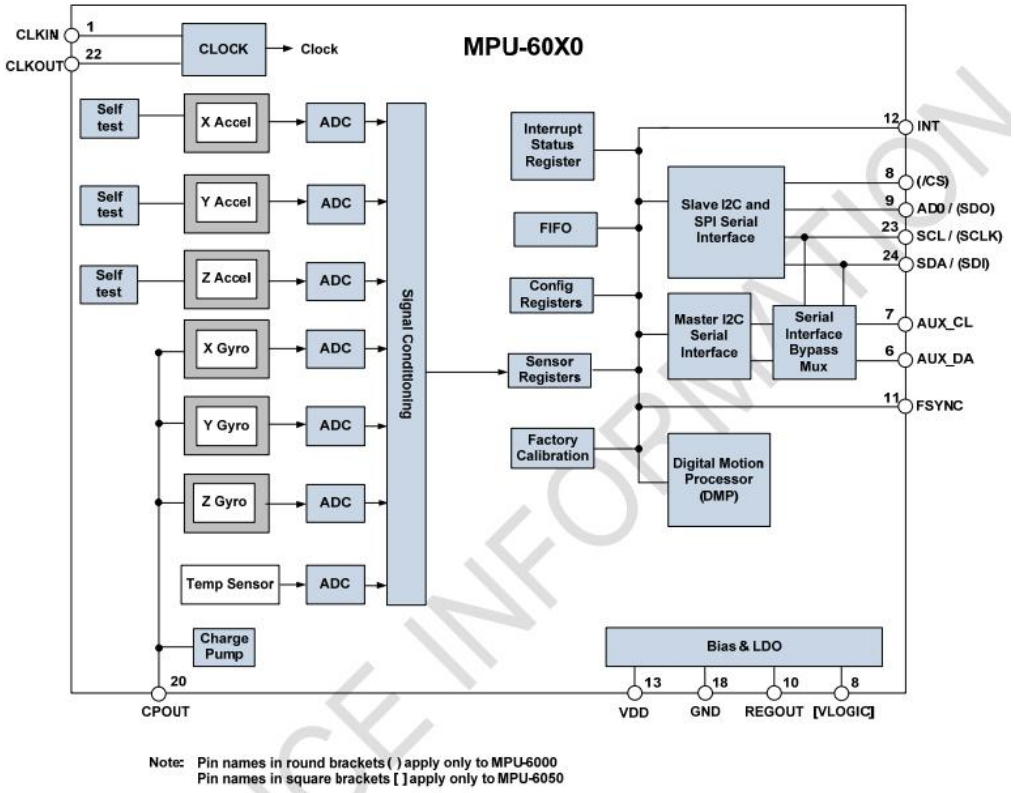


图 5.1.3 MPU-6050 内部框图

图 5.1.3 中，SCL 和 SDA 是连接 MCU 的 IIC 接口，MCU 通过这个 IIC 接口来控制 MPU6050，另外还有一个 IIC 接口：AUX_CL 和 AUX_DA，这个接口可用来连接外部从设备，比如磁传感器，这样就可以组成一个九轴传感器。VLOGIC 是 IO 口电压，该引脚最低可以到 1.8V，我们一般直接接 VDD 即可。AD0 是从 IIC 接口（接 MCU）的地址控制引脚，该引脚控制 IIC 地址的最低位。如果接 GND，则 MPU6050 的 IIC 地址是：0X68，如果接 VDD，则是 0X69，注意：这里的地址是不包含数据传输的最低位的（最低位用来表示读写）！！

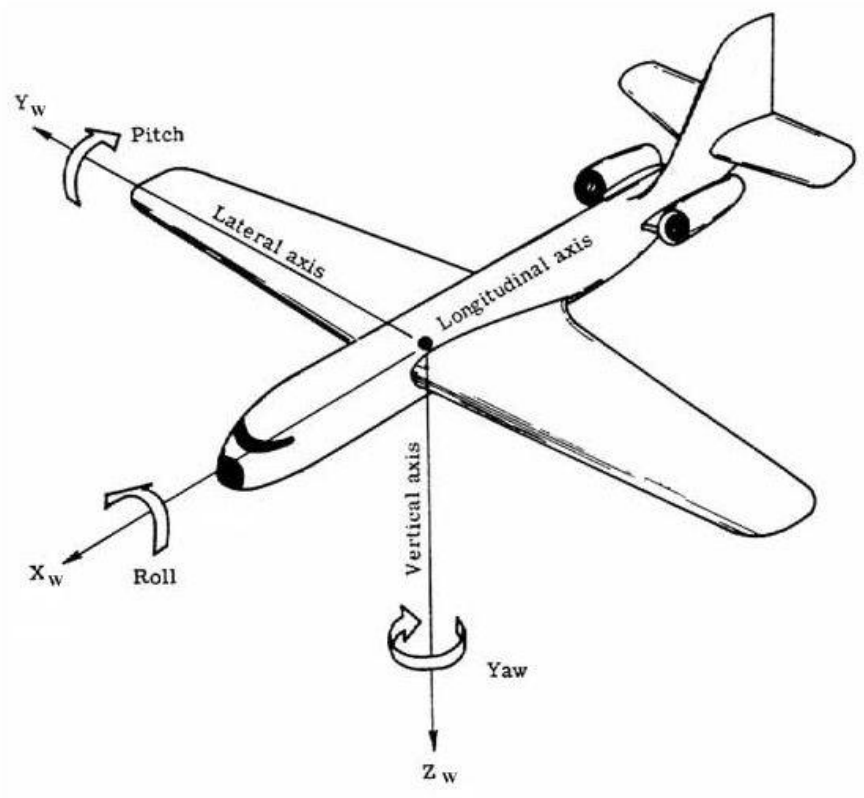


图 5.1.4 姿态角示意图

2、三轴加速度计/陀螺仪原理

加速度计：质量块 + 差分电容

如图 5.2.1-5.2.3 所示，芯片里有一组可动的硅质量块，加速度让质量块位移，改变它与固定极板之间的电容，检测电容差即得加速度。关键点：静止时能测出重力方向（Z 轴读数约 1 g），所以可以用它算 Roll/Pitch；但一旦有运动加速度或振动，读数就不再是纯重力，算出的角度会抖。

陀螺仪：科里奥利力

内部有持续振动的质量块，当芯片绕某轴旋转时，振动质量块受到科里奥利力，产生垂直于振动方向的位移，检测该位移即得角速度。关键点：它测的是角速度（°/s），要积分才得到角度，而积分会把零偏误差累积——这就是漂移的根源。

一句话总结：加速度计是"看方向"的（长期准、短期抖），陀螺仪是"记过程"的（短期准、长期漂）。两个都不完美，合起来才好用——这就是互补滤波的动机。

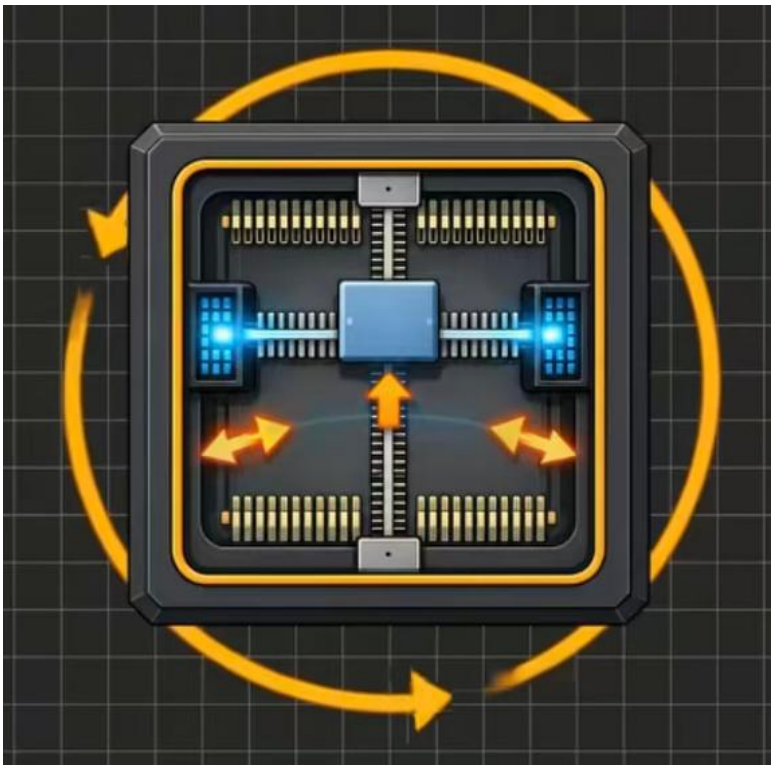


图 5.1.1 MEMS 传感器的工作原理 1

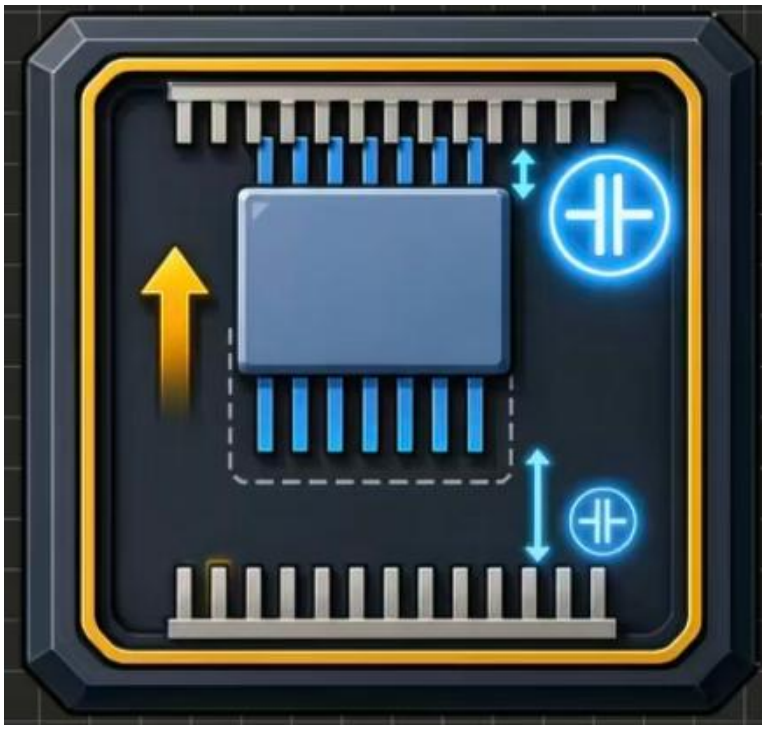


图 5.1.2 MEMS 传感器的工作原理 2

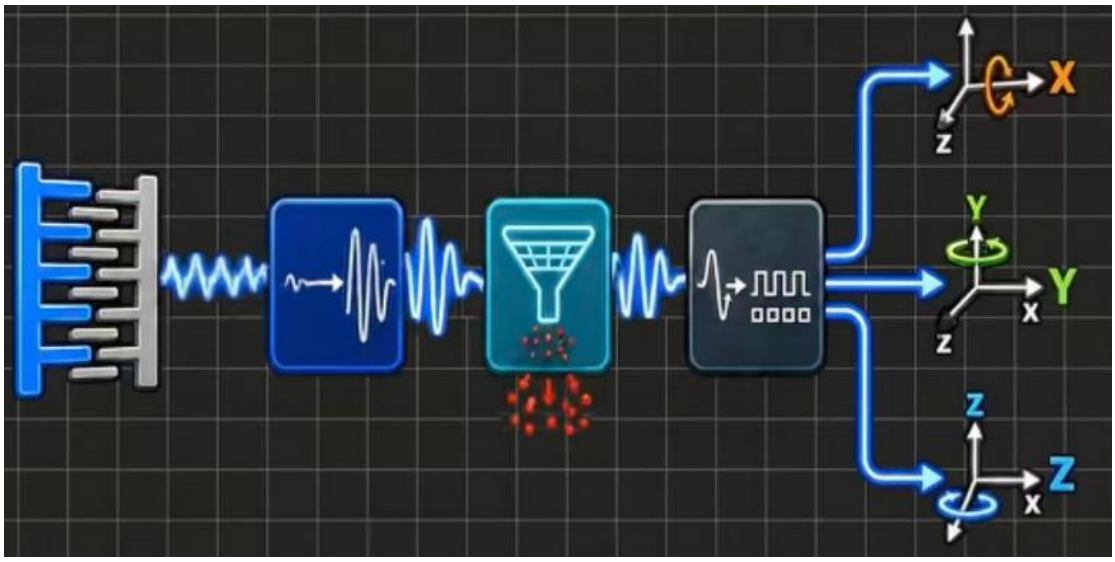


图 5.1.3 微小电容检测电路

3、关键寄存器速查表（最好配合 MPU6050 手册看）

地址	寄存器	作用与推荐值
0x19	SMPLRT_DIV	采样分频。输出频率 = 1 kHz ÷ (DIV+1)，DIV=9 → 100 Hz
0x1A	CONFIG	DLPF 数字低通。写 0x06（带宽约 5 Hz，最抗抖）
0x1B	GYRO_CONFIG	陀螺仪量程。0x00 = ±250 °/s（131 LSB）
0x1C	ACCEL_CONFIG	加速度计量程。0x00 = ±2 g（16384 LSB/g）
0x3B	ACCEL_XOUT_H	**从 0x3B-0x40 存贮三轴加速度测量值**，高位在前，低位在后
0x41/0x42	TEMP_OUT_H	温度高 8 位，温度低 8 位
0x43	GYRO_XOUT_H	**从 0x43-0x48 存放**陀螺仪数据，高 8 位在前
0x6B	PWR_MGMT_1	**电源管理**。写 0x80 复位，写 0x00 唤醒（务必做）
0x75	WHO_AM_I	**器件 ID**，正常返回 0x68。用来验证通信是否通

从 0x3B 开始的连续 14 字节存放的是加速度计、温度、陀螺仪的数据，因此可以采取突发读取的方式一次全部读出，14 字节数据数据块的排列：加速度 X/Y/Z（6 字节）→ 温度（2 字节）→ 陀螺 X/Y/Z（6 字节），每个轴都是高字节在前。一次突发读 14 字节最省事，比逐个寄存器读快得多。

4、硬件连接

注意：STM32F103 的硬件 I2C 引脚不包含 PB14 / PB15。所以本设计里必须、也只能使用软件模拟 I2C 时序。

MPU6050 模块引脚	STM32F103RCT6	说明
VCC	5 V	模块供电脚
GND	GND	共地
SCL	**PB14**	软件模拟时钟线，建议**开漏输出模式**
SDA	**PB15**	软件模拟数据线，建议**开漏输出模式**
AD0	4.7K 接地	接地 → 器件地址 0x68；接 3.3 V → 0x69
INT	PB9	中断引脚

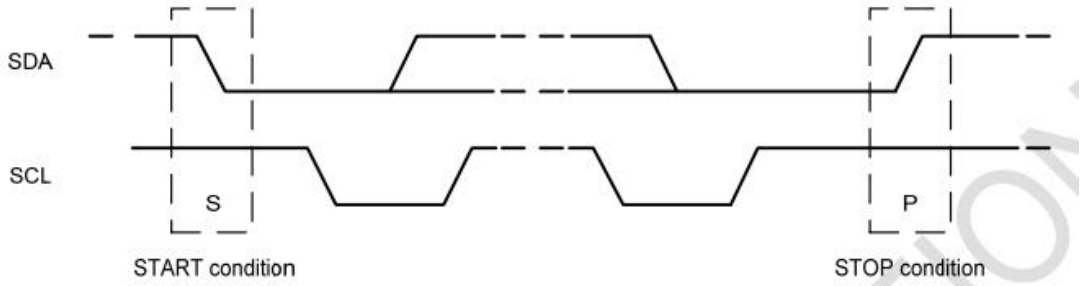
为什么要用开漏输出（GPIO_MODE_OUTPUT_OD）

I2C 是"线与"总线：任何器件都能把线拉低，谁都不拉时由上拉电阻拉高。如果配置成推挽输出，主机输出高电平会与从机拉低硬碰硬，形成大电流。开漏模式下主机输出 1 实际是"释放总线"，交给上拉电阻，总线才安全。模块已带 4.7 kΩ 上拉，一般不用外接。

安装位置的讲究：MPU6050 要装在车体重心附近、远离电机的位置，并用螺丝刚性固定（别用双面胶软连接）。电机振动会直接污染加速度计读数，这是很多"数据乱跳"的真正原因，不是代码问题。

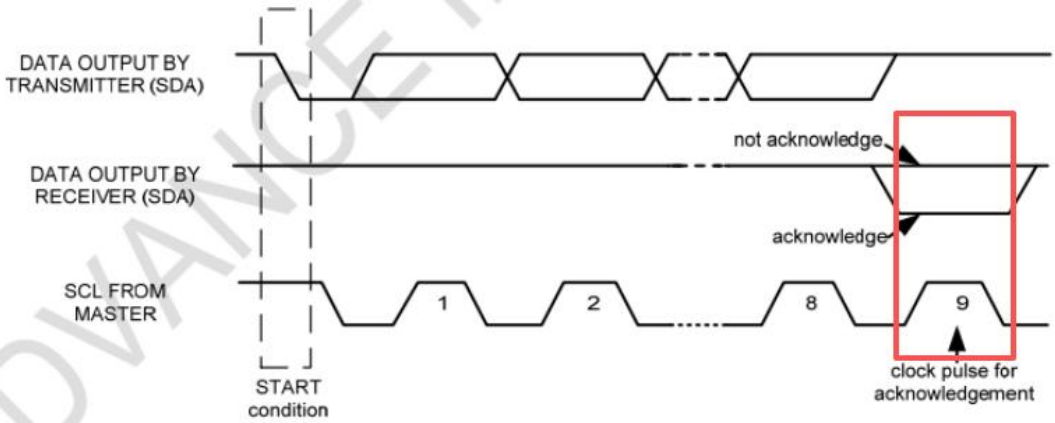
5、软件 I2C 时序与代码

I2C 通讯主要有四个基本动作：起始信号 S、停止信号 P、应答 ACK、发字节，分别见图 5.5.1-5.5.4。



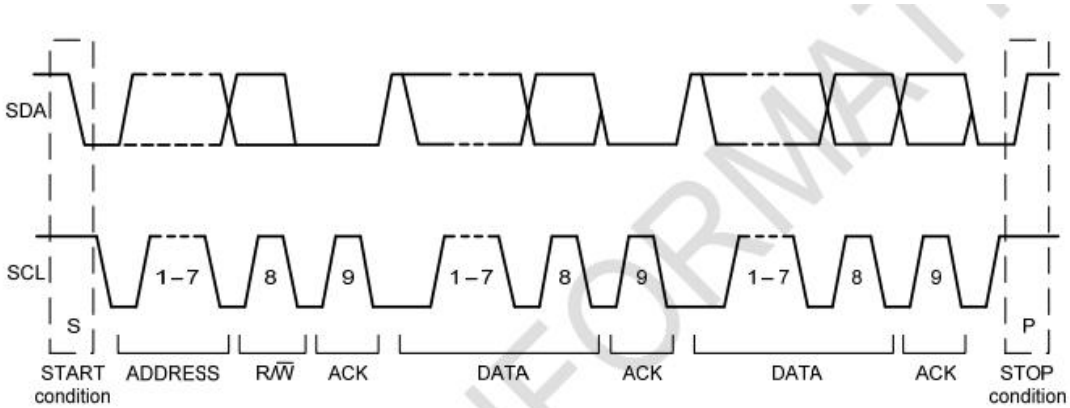
START and STOP Conditions

图 5.5.1 起始信号和停止信号



Acknowledge on the I²C Bus

图 5.5.2 应答信号



Complete I²C Data Transfer

图 5.5.3 数据发送

6、MPU-6050 基本驱动

6.1 MPU6050 初始化

单片机读取 MPU6050 的加速度和角度传感器数据，需要以下初始化步骤：

1）初始化 IIC 接口

MPU6050 采用 IIC 与 STM32F103 通信，需要先初始化与 MPU6050 连接的 SDA 和 SCL 数据线（PB14/PB15/PB9）。

2）复位 MPU6050

这一步让 MPU6050 内部所有寄存器恢复默认值，通过对电源管理寄存器 1（0X6B）的 bit7 写 1 实现。 复位后，电源管理寄存器 1 恢复默认值(0X40)，然后必须设置该寄存器为 0X00，以唤醒 MPU6050，进入正常工作状态。

3）配置系统时钟源

系统时钟源同样是通过电源管理寄存器 1（0X6B）来设置，该寄存器的最低三位用于设置系统时钟源选择，默认值是 0（内部 8M RC 震荡），不过我们一般设置为 1 or 2，选择 x or y 轴陀螺 PLL 作为时钟源，以获得更高精度的时钟。

4）设置角速度传感器（陀螺仪）和加速度传感器的满量程范围

设置两个传感器的满量程范围(FSR)，分别通过陀螺仪配置寄存器（0X1B）和加速度传感器配置寄存器（0X1C）设置。我们一般设置陀螺仪的满量程范围为±2000dps，加速度传感器的满量程范围为±

2g。同时，使能角速度传感器和加速度传感器，这两个操作通过电源管理寄存器 2（0X6C）来设置，设置对应位为 0 即可开启。

5）设置其他参数

还需要配置的参数有：中断、AUX IIC 接口、FIFO、陀螺仪采样率和设置数字低通滤波器（DLPF）等。由于采用中断方式读取数据，所以开中断，系统没有外加磁力计，关闭 AUX IIC 接口。分别通过中断使能寄存器（0X38）和用户控制寄存器（0X6A）控制。根据使用的传感器，通过 FIFO 使能寄存器（0X23）控制对应的 FIFO 开启。陀螺仪采样率通过采样率分频寄存器（0X19）控制，这个采样率我们一般设置为 50 即可。数字低通滤波器(DLPF)则通过配置寄存器（0X1A）设置，一般设置 DLPF 为带宽的 1/2 即可。

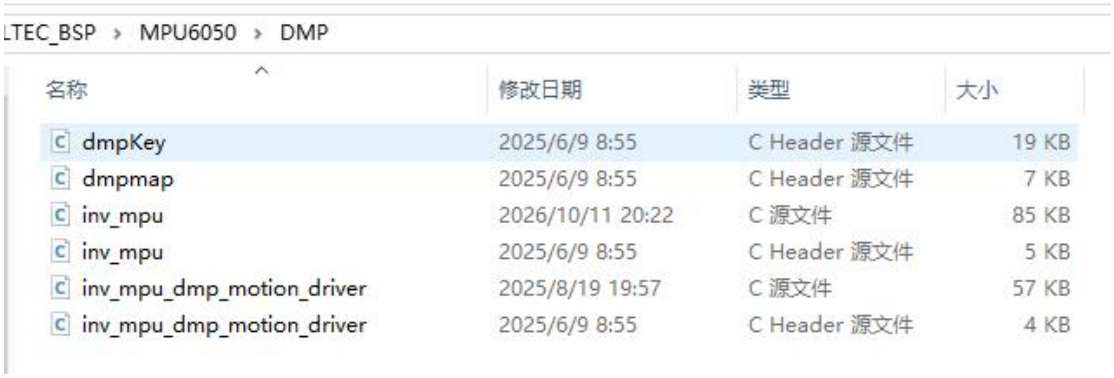
至此，MPU6050 的初始化就完成了，可以正常工作了（其他未设置的寄存器全部采用默认值即可），接下来，我们就可以读取相关寄存器，得到加速度传感器、角速度传感器和温度传感器的原始数据了。

6.2 DMP 初始化

经过 MPU6050 的初始化后虽然可以读出 MPU6050 的加速度传感器和角速度传感器的原始数据，但这些数据对做平衡车、搞四轴之类的来说用处不大，我们期望得到的是姿态数据，也就是欧拉角：航向角（yaw）、横滚角（roll）和俯仰角（pitch）。要得到欧拉角数据，就得利用原始数据，进行姿态融合解算，这个比较复杂，对单片机开销极大，初学者也不易掌握。而 MPU6050 自带了数字运动

处理器，即 DMP，专门独立处理 6 轴原始数据，完全不用 MCU 做姿态解算，大大降低了 MCU 的负担，从而有更多的时间去处理其他事件，提高系统实时性。MCU 只需要通过 I2C 读取 DMP 输出好的姿态结果（四元数 / 欧拉角）。

DMP 库包含下图中的 6 个文件，重点就是两个 c 文件：inv_mpu.c 和 inv_mpu_dmp_motion_driver.c。



DMP 的初始化流程图如图 6.2.1 所示：



图 6.2.1 DMP 初始化流程

下面简单介绍几个重要的寄存器。

1）电源管理寄存器 1，该寄存器地址为 0X6B，各位描述如下表

格所示：

Register (Hex)	Register (Decimal)	Bit7	Bit6	Bit5	Bit4	Bit3	Bit2	Bit1	Bit0
6B	107	DEVICE _RESET	SLEEP	CYCLE	-	TEMP_DIS	CLKSEL[2:0]		

其中，DEVICE_RESET 位用来控制复位，设置为 1，复位 MPU6050，复位结束后，MPU 硬件自动清零该位。SLEEP 位用于控制 MPU6050 的工作模式，复位后，该位为 1，即进入了睡眠模式(低功耗)，所以我们要清零该位，以进入正常工作模式。TEMP_DIS 用于设置是否使能温度传感器，设置为 0，则使能。最后 CLKSEL[2:0] 用于选择系统时钟源，选择关系下表所示：

CLKSEL[2:0]	时钟源
000	内部 8M RC 晶振
001	PLL，使用 X 轴陀螺作为参考
010	PLL，使用 Y 轴陀螺作为参考
011	PLL，使用 Z 轴陀螺作为参考
100	PLL，使用外部 32.768Khz 作为参考
101	PLL，使用外部 19.2Mhz 作为参考
110	保留
111	关闭时钟，保持时序产生电路复位状态

默认是使用内部 8M RC 晶振的，精度不高，所以我们一般选择 X/Y/Z 轴陀螺作为参考的 PLL 作为时钟源，一般设置 CLKSEL=001 即可。

2）陀螺仪配置寄存器，该寄存器地址为：0X1B，各位描述如下表所示：

Register (Hex)	Register (Decimal)	Bit7	Bit6	Bit5	Bit4	Bit3	Bit2	Bit1	Bit0
1B	27	XG_ST	YG_ST	ZG_ST	FS_SEL[1:0]		-	-	-

该寄存器我们只关心 FS_SEL[1:0]这两个位，用于设置陀螺仪的满量程范围：0，±250°/S；1，±500°/S；2，±1000°/S；3，±2000°/S；一般设置为 3，即±2000°/S，因为陀螺仪的 ADC 为 16 位分辨率，所以得到灵敏度为：65536/4000=16.4LSB/(°/S)。

3）加速度传感器配置寄存器，寄存器地址为：0X1C，各位描述如下表所示：

Register (Hex)	Register (Decimal)	Bit7	Bit6	Bit5	Bit4	Bit3	Bit2	Bit1	Bit0
1C	28	XA_ST	YA_ST	ZA_ST	AFS_SEL[1:0]		-		

该寄存器我们只关心 AFS_SEL[1:0]这两个位，用于设置加速度传感器的满量程范围：0，±2g；1，±4g；2，±8g；3，±16g；我们一般设置为 0，即±2g，因为加速度传感器的 ADC 也是 16 位，所以得到灵敏度为：65536/4=16384LSB/g。

4）FIFO 使能寄存器，寄存器地址为：0X23，各位描述如下表所示：

Register (Hex)	Register (Decimal)	Bit7	Bit6	Bit5	Bit4	Bit3	Bit2	Bit1	Bit0
23	35	TEMP_ FIFO_EN	XG_ FIFO_EN	YG_ FIFO_EN	ZG_ FIFO_EN	ACCEL _FIFO_EN	SLV2 _FIFO_EN	SLV1 _FIFO_EN	SLV0 _FIFO_EN

该寄存器用于控制 FIFO 使能，在简单读取传感器数据的时候，可以不用 FIFO，设置对应位为 0 即可禁止 FIFO，设置为 1，则使能 FIFO。注意加速度传感器的 3 个轴，全由 1 个位（ACCEL_FIFO_EN）控制，只要该位置 1，则加速度传感器的三个通道都开启 FIFO 了。

5）陀螺仪采样率分频寄存器，寄存器地址为：0X19，各位描述如下表所示：

Register (Hex)	Register (Decimal)	Bit7	Bit6	Bit5	Bit4	Bit3	Bit2	Bit1	Bit0
19	25	SMPLRT_DIV[7:0]							

该寄存器用于设置 MPU6050 的陀螺仪采样频率，计算公式为：

采样频率 = 陀螺仪输出频率 / (1+SMPLRT_DIV)

这里陀螺仪的输出频率，是 1Khz 或者 8Khz，与数字低通滤波器 (DLPF) 的设置有关，当 DLPF_CFG=0/7 的时候，频率为 8Khz，其他情况是 1Khz。而且 DLPF 滤波频率一般设置为采样率的一半。采样率，我们假定设置为 50Hz，那么 MPLRT_DIV=1000/50-1=19。

6）配置寄存器，寄存器地址为：0X1A，各位描述如下表所示：

Register (Hex)	Register (Decimal)	Bit7	Bit6	Bit5	Bit4	Bit3	Bit2	Bit1	Bit0
1A	26	-	-	EXT_SYNC_SET[2:0]			DLPF_CFG[2:0]		

这里，我们主要关心数字低通滤波器（DLPF）的设置位，即：DLPF_CFG[2:0]，加速度计和陀螺仪都是根据这三个位的配置进行过滤的。DLPF_CFG 不同配置对应的过滤情况如下表所示：

DLPF_CFG[2:0]	加速度传感器 Fs=1Khz		角速度传感器（陀螺仪）		
	带宽(Hz)	延迟（ms）	带宽(Hz)	延迟（ms）	Fs(Khz)
000	260	0	256	0.98	8
001	184	2.0	188	1.9	1
010	94	3.0	98	2.8	1
011	44	4.9	42	4.8	1
100	21	8.5	20	8.3	1
101	10	13.8	10	13.4	1
110	5	19.0	5	18.6	1
111	保留		保留		8

这里的加速度传感器，输出速率（Fs）固定是 1Khz，而角速度传感器的输出速率（Fs），则根据 DLPF_CFG 的配置有所不同。一般我们设置角速度传感器的带宽为其采样率的一半，如前面所说的，如果设置采样率为 50Hz，那么带宽就应该设置为 25Hz，取近似值 20Hz，就应该设置 DLPF_CFG=100。

7）电源管理寄存器 2，寄存器地址为：0X6C，各位描述如下表所示：

Register (Hex)	Register (Decimal)	Bit7	Bit6	Bit5	Bit4	Bit3	Bit2	Bit1	Bit0
6C	108	LP_WAKE_CTRL[1:0]		STBY_XA	STBY_YA	STBY_ZA	STBY_XG	STBY_YG	STBY_ZG

该寄存器的 LP_WAKE_CTRL 用于控制低功耗时的唤醒频率，暂时用不到。剩下的 6 位，分别控制加速度和陀螺仪的 x/y/z 轴是否进入待机模式，这里我们全部都不进入待机模式，所以全部设置为 0 即可。

8）陀螺仪数据输出寄存器，总共有 6 个寄存器组成，地址为：0X43~0X48，通过读取这 6 个寄存器，就可以读到陀螺仪 x/y/z 轴的值，比如 x 轴的数据，可以通过读取 0X43（高 8 位）和 0X44（低 8 位）寄存器得到，其他轴以此类推。

9）加速度传感器数据输出寄存器，也有 6 个，地址为：0X3B~0X40，通过读取这 6 个寄存器，就可以读到加速度传感器 x/y/z 轴的值，比如读 x 轴的数据，可以通过读取 0X3B（高 8 位）和 0X3C（低 8 位）寄存器得到，其他轴以此类推。

10）温度传感器的值，可以通过读取 0X41（高 8 位）和 0X42（低 8 位）寄存器得到，温度换算公式为：

$$Temperature = 36.53 + regval/340$$

其中，Temperature 为计算得到的温度值，单位为℃，regval 为从 0X41 和 0X42 读到的温度传感器值。

7、姿态解算与互补滤波

7.1 方法一：只用加速度计（静态可用，动态差）

```
/* 单位：度 */
acc_roll = atan2f(ay, az) * 57.29578f;//弧度转角度*57.3
acc_pitch = atan2f(-ax, az) * 57.29578f;
```

原理：静止时加速度计只感受重力，重力在三轴上的投影比例就反映了倾斜角。缺点：一有振动或加减速，读数里混进运动加速度，角度立刻跳。

7.2 方法二：只用陀螺仪（动态好，必然漂）

```
yaw += gz * dt; /* gz 单位 °/s，dt 单位 s */
roll += gx * dt;
pitch += gy * dt;
```

原理：角速度积分成角度。缺点：零偏（静止时不为 0 的那个小读数）会被积分累积，几秒钟就能漂出好几度。

7.3 方法三：互补滤波

方法一和方法二都有局限，针对二者的缺点（加速度计解算怕振动，只用陀螺仪解算怕长期累积漂移）把低通滤波（只信任低频（缓慢变化）的姿态）和高通滤波（只信任短期快速变化的姿态）结合起来，效果不错。图 7.1 是互补滤波的框图，权重根据不同场景可调。

振动大场景：降低加速度计权重，例如 0.99 / 0.01；

漂移严重场景：适当加大加速度计权重， 0.97 / 0.03。



图 7.1 · 互补滤波：把"长期准"和"短期准"互补

```
#define ALPHA 0.98f /* 陀螺权重，越大越信陀螺 */

void Attitude_Update(MPU_Raw_t *raw, float gxo, float gyo, float gzo, float dt)
{
   /* 1. 换算并以 g / °/s 为单位 */
   float ax = raw->ax / 16384.0f;
   float ay = raw->ay / 16384.0f;
   float az = raw->az / 16384.0f;
   float gx = raw->gx / 131.0f - gxo; /* 减零偏！ */
   float gy = raw->gy / 131.0f - gyo;
   float gz = raw->gz / 131.0f - gzo;

   /* 2. 加速度计解算角(度) */
   float acc_roll = atan2f(ay, az) * 57.29578f;
   float acc_pitch = atan2f(-ax, az) * 57.29578f;

   /* 3. 互补滤波：陀螺积分为主，加速度慢慢把它拉回来 */
   g_roll = ALPHA * (g_roll + gx * dt) + (1.0f - ALPHA) * acc_roll;
   g_pitch = ALPHA * (g_pitch + gy * dt) + (1.0f - ALPHA) * acc_pitch;
   g_yaw += gz * dt; /* Yaw 无绝对参考，只能积分 */
}
```

零偏校准：上电后车体静止不动，连续采样 200 次陀螺数据取平均，作为 gxo/gyo/gzo 存起来，之后每次读数都减掉。不做这步，Yaw 一分钟能漂十几度。

```
void MPU6050_Calibrate(float *gxo, float *gyo, float *gzo)
```

```
{
  int32_t sx = 0, sy = 0, sz = 0;
  MPU_Raw_t r;
  for (int i = 0; i < 200; i++)
  {
    MPU6050_ReadRaw(&r);
    sx += r.gx; sy += r.gy; sz += r.gz;
    Delay_ms(5);
  }
  *gxo = (float)sx / 200.0f / 131.0f;
  *gyo = (float)sy / 200.0f / 131.0f;
  *gzo = (float)sz / 200.0f / 131.0f;
}
```

7.4 方法四：DMP（Digital Motion Processor）

MPU6050 的 DMP 输出的四元数是 q30 格式的，也就是浮点数放大了 2 的 30 次方倍。在换算成欧拉角之前，必须先将其转换为浮点数，也就是除以 2 的 30 次方，然后再进行计算，计算公式为：

q0=quat[0] / q30; //q30 格式转换为浮点数

q1=quat[1] / q30;

q2=quat[2] / q30;

q3=quat[3] / q30;

//计算得到俯仰角/横滚角/航向角

pitch=asin(-2 * q1 * q3 + 2 * q0* q2)* 57.3; //俯仰角

roll=atan2(2 * q2 * q3 + 2 * q0 * q1, -2 * q1 * q1 - 2 * q2* q2 + 1)* 57.3; //横滚角

yaw=atan2(2*(q1*q2 + q0*q3),q0*q0+q1*q1-q2*q2-q3*q3) * 57.3; //航向角

其中 quat[0]~ quat[3]是 MPU6050 的 DMP 解算后的四元数，q30 格式，所以要除以一个 2 的 30 次方，其中 q30 是一个常量：1073741824，即 2 的 30 次方，然后带入公式，计算出欧拉角。上述计算公式的 57.3 是弧度转换为角度，即 180/π，这样得到的结果就是以度（°）为单位的。

【注】DMP 输出的四元数已经保证模长等于 1，不需要再做归一化处理。

其他方法 Mahony 滤波（改进互补滤波）、Madgwick 滤波（梯度下降法）、卡尔曼滤波等。

8、Yaw 漂移的成因与对策

由于 MPU6050 不带磁力计，单靠 MPU6050 不可能得到长期准确的 Yaw。因为重力方向只能提供 Roll/Pitch 的绝对参考，Z 轴（水平旋转）在重力场里没有可参照的绝对方向，只能靠陀螺积分，积分必漂。

对策	效果	说明
做好零偏校准	减缓明显	能把漂移从每分钟 10° 压到 1° 量级
温度补偿	有一定效果	零偏随温度变，可建立温度-零偏表

对策	效果	说明
加磁力计	**根本解决**	**HMC5883L / QMC5883，靠地磁提供绝对航向**
编码器 / 轮速辅助	短期很有效	麦轮车本身有编码器，用里程计修正 Yaw
短时间使用后重置	实用土办法	每次动作开始前把 yaw 归零，只关心相对转角

麦轮车最常见的需求是"走直线不跑偏"和"转 90° 要准"。这两个场景都不需要绝对航向——每次动作前把 yaw 清零，只控制相对转角，漂移的影响就被限制在单次动作时间内（几秒），完全够用。

9、Yaw 闭环：把姿态真正用起来

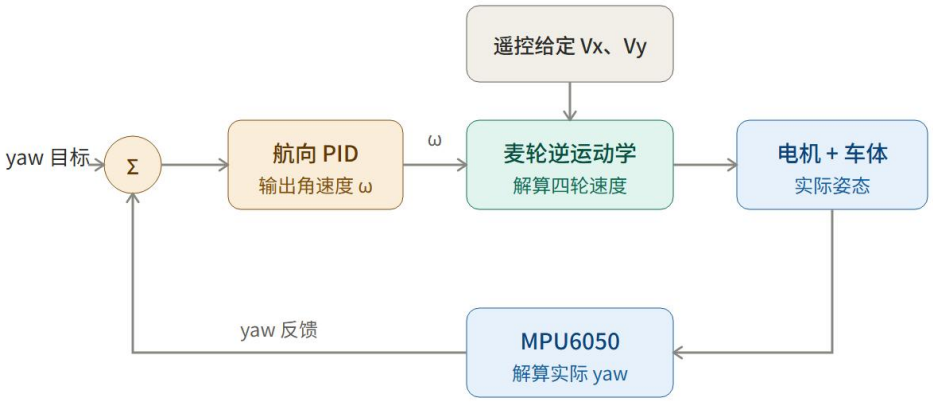


图 9.1 · 小车航向闭环控制

为什么麦轮车特别需要这个闭环?

麦轮车横移和斜走时，四个轮子受力方向和地面摩擦很难完全一致，加上轮径公差、电机特性差异，车会一边走一边慢慢转头。开环

控制下"让它直行"基本做不到。加入 Yaw 闭环后，车头一偏就被 PID 拉回来，直线行走才无压力。

10、程序解读

C10B 工程的 MPU6050 不走"读原始 ADC + 自己滤波积分"的路子，而是启用芯片内部的 DMP（Digital Motion Processor）：把 InvenSense 官方的 3062 字节运动处理固件烧进 MPU6050 的 DMP 内存，由 DMP 在硬件里完成姿态融合，直接通过 FIFO 吐出 Q30 格式的四元数。

STM32 单片机只做两件事：I2C 取包、四元数换算成 Roll/Pitch/Yaw，最后送 OLED 显示。

触发方式：DMP 每 10ms（100Hz）在 INT 引脚（PB9）输出一个下降沿，STM32 用它作为整个系统的 100Hz 时基——不仅读姿态，还顺带驱动超声波（20Hz）、温度采样（0.2Hz）、K210 掉线检测（1s）、向 C50C 底盘上报（20Hz）。

10.1 程序分层架构

应用层	main.c main() ——▶ MPU6050_initialize() / DMP_Init() EXTI 中断 ——▶ Read_DMP() ——▶ Roll/Pitch/Yaw ——▶ OLED / UART5
驱动层	MPU6050.c / MPU6050.h （WHEELTEC 二次封装） IICwriteBits / IICwriteBit / IICreadBytes / i2cRead MPU6050_initialize / DMP_Init / Read_DMP / Read_Temperature
通讯层	inv_mpu.c （InvenSense 官方驱动） mpu_init / mpu_set_sensors / mpu_configure_fifo mpu_set_sample_rate / mpu_write_mem / mpu_load_firmware mpu_set_dmp_state / mpu_read_fifo_stream / mpu_run_self_test
固件层	inv_mpu_dmp_motion_driver.c dmp_memory[3062] ← DMP 固件镜像（写进芯片 DMP 内存） dmp_load_motion_driver_firmware / dmp_set_orientation dmp_enable_feature / dmp_set_fifo_rate / dmp_read_fifo
板级层	bsp_siic.c （软件模拟 I2C，PB14=SCL / PB15=SDA） Start/Stop/WaitAck/SendByte/ReadByte IIC_Mem_Read / IIC_Mem_Write ——▶ User_sIICDev 函数指针表

10.2 涉及文件清单

#	文件	行数	所在层	作用
1	WHEELTEC_BSP\MPU6050\MPU6050.h	396	驱动层（对外接口）	寄存器地址/位定义宏、Imu_t 结构体、对外 API 声明
2	WHEELTEC_BSP\MPU6050\MPU6050.c	429	驱动层	位域读写、寄存器配置、DMP_Init、Read_DMP、温度读取
3	WHEELTEC_BSP\MPU6050\DMP\inv_mpu.c	2885	通讯层	寄存器级读写封装、量程/采样率/FIFO/内存读写、DMP 使能
4	WHEELTEC_BSP\MPU6050\DMP\inv_mpu.h	124	通讯层	mpu_* API 声明、传感器掩码、中断状态位
5	WHEELTEC_BSP\MPU6050\DMP\inv_mpu_dmp_motion_driver.c	1367	固件层	DMP 固件二进制数组（3062B）+ DMP 功能配置/解析

#	文件	行数	所在层	作用
6	WHEELTEC_BSP\MPU6050\DMP\inv_mpu_dmp_motion_driver.h	92	固件层	DMP 功能掩码（DMP_FEATURE_*）、dmp_* API 声明
7	WHEELTEC_BSP\MPU6050\DMP\dmpKey.h	494	固件层	DMP 内存"逻辑键"→"物理地址"的换算宏
8	WHEELTEC_BSP\MPU6050\DMP\dmpmap.h	264	固件层	DMP 内存变量表（DMP_XGYR、DMP_FIFORT…）

外部耦合文件：

文件	涉及内容
WHEELTEC_BSP\bsp_siic.c/.h	软件模拟 I2C 的物理实现 + IICInterface_t 函数指针接口
Core\Src\main.c	调用 MPU6050_initialize() / DMP_Init()；HAL_GPIO_EXTI_Callback() 里调 Read_DMP()
Core\Src\gpio.c	PB9 配置成下降沿外部中断，上拉
Core\Src\stm32f1xx_it.c	EXTI9_5_IRQHandler() 把中断转给 HAL
Core\Inc\main.h	MPU6050_INT_Pin/Port/IRQn 宏定义

10.3 运行流程

初始化时序（从上电到 INT 开始跳）

main()	
├── HAL_Init() / SystemClock_Config()	HCLK 72MHz / PCLK1 36MHz / PCLK2 72MHz
├── MX_GPIO_Init()	PB14/PB15 输出；PB9 = IT_FALLING + 上拉
├── MX_DMA_Init() / UART5 / USART3 / TIM2 / TIM7 / USART1	
├── DWT_Init()	delay_us/delay_ms 的时基
├── OLED_Init()	
├── HAL_TIM_IC_Start_IT(&htim2, CH2)	超声波捕获就绪
├── MPU6050_initialize()	查 ID；② 设 Y-PLL/±2000dps/±2g/唤醒（参数随后被 mpu_init 覆盖，仅起探活作用）
└── DMP_Init()	
├── i2cRead(0x68, 0x75)	再查 ID
├── mpu_init()	★ 软复位 0x80 → 100ms → 唤醒
|	读 rev → 配 2000dps/2g/LPF42/50Hz
├── mpu_set_sensors(GYRO|ACCEL)	PWR_MGMT_1 = PLL，PWR_MGMT_2 = 0，延时 50ms

├── mpu_configure_fifo(...)	打开 data-ready 中断
├── mpu_set_sample_rate(100)	SMPLRT_DIV = 9，LPF 自动 → 42Hz
├── dmp_load_motion_driver_firmware()	★ 3062B 分 192 包写入 + 回读校验（≈190ms）
├── dmp_set_orientation(0x0A4)	diag(-1,-1,1) 绕 Z 转 180°
├── dmp_enable_feature(...)	6+6+16+4 = 32B/包
├── dmp_set_fifo_rate(100)	div = 1
├── run_self_test()	自检 + 零偏（本工程通常不满足 0x7，跳过）
└── mpu_set_dmp_state(1)	★ INT_ENABLE = 0x02，FIFO 复位 ⟹PB9 开始 100Hz 下降沿
main() while(1) ——► OLED 刷新 Pitch/Roll/Yaw/temp/dis/cx/size ， delay_ms(10)	

中断驱动的数据流（核心）

MPU6050 DMP 算完一包 (每 10ms)	
INT 引脚 (PB9) 下降沿	
EXTI9_5_IRQHandler()	stm32f1xx_it.c
HAL_GPIO_EXTI_IRQHandler(PB9)	清挂起标志 → 回调
HAL_GPIO_EXTI_Callback(PB9)	main.c:265
├── temperature_cnt++	每 500 次(5s)：读 0x41 两字节 → temperature
├── Read_DMP()	I2C 读 FIFO 一包(32B) → 四元数 → Roll/Pitch/Yaw
├── ranger_cnt++	每 5 次(50ms)：PC15(ranger_tri) 发 15us 高脉冲触发超声波
├── k210_active_cnt++	每 100 次(1s)：置 k210_active_flag = 0（掉线）
└── c50c_send_cnt++	每 5 次(20Hz)：组 12 字节帧 → UART5 阻塞发给 C50C
Read_DMP() 内部： dmp_read_fifo(gyro, accel, quat, &ts, &sensors, &more)	
└── mpu_read_fifo_stream(32, buf, more)	
├── i2c_read(0x72, 2)	读 FIFO_COUNTH/L
├── [必要时] i2c_read(0x3A,1) 读 INT_STATUS 查溢出	

└── i2c_read(0x74, 32)	读 FIFO_R_W 32 字节
└──解析→quat[4] (Q30) → sensors |= INV_WXYZ_QUAT	
→ q0..q3 = quat/2^30	
→ Roll/Pitch/Yaw 全局变量	

10.4 分层源码解析

10.4.1 板级层：软件模拟 I2C（bsp_siic.c）

定义"函数指针结构体"，实现上层代码与具体 I2C 实现解耦：OLED 和 MPU6050 都用 I2C，但以后可能一个用硬件 I2C、一个用软件 I2C。于是定义了统一的接口结构（bsp_siic.h）：

```
typedef struct {
    void (*init)(void);

    /* 基础收发：不带寄存器地址（少数器件用） */
    IIC_Status_t (*write)(uint16_t DevAddress, uint8_t *pData, uint16_t Size,
uint32_t Timeout);
    IIC_Status_t (*read)(uint16_t DevAddress, uint8_t *pData, uint16_t Size,
uint32_t Timeout);

    /* 带寄存器读写：传感器类器件的标准玩法（先发寄存器地址，再收发数据） */
    IIC_Status_t (*write_reg)(uint16_t DevAddress, uint16_t MemAddress,
uint8_t *pData, uint16_t Size, uint32_t Timeout);
    IIC_Status_t (*read_reg)(uint16_t DevAddress, uint16_t MemAddress, uint8_t
*pData, uint16_t Size, uint32_t Timeout);

    void (*delay_ms)(uint16_t ms);
} IICInterface_t, *pIICInterface_t;

extern IICInterface_t User_sIICDev;   /* 全局的"驱动实例" */
```

实现与软件 I2C 的 6 个静态函数关联：

```
/* 挂载驱动（填充全局接口实例）
 * 调用方只需持有 IICInterface_t* 指针，不关心底层是软件还是硬件 I2C */
IICInterface_t User_sIICDev = {
    .init     = siic_init,           /* 空函数，引脚已在 MX_GPIO_Init 里配好 */
    .write    = IIC_Master_Transmit,
    .read     = IIC_Master_Receive,
    .write_reg= IIC_Mem_Write,       /* MPU6050 实际用的就是这两个 */
```

```
    .read_reg = IIC_Mem_Read,
    .delay_ms = iic_delayms
};
```

I2C 时序实现要点:

```
/* 起始信号：SCL 高电平期间，SDA 由高变低 */
static void sIIC_Start(void)
{
    SDA_OUT();          /* SDA 必须切回输出模式才能主动拉电平 */
    sIIC_SCL_L;
    sIIC_SDA_H;         /* 先把 SDA 拉高 */
    sIIC_SCL_H;         /* 再拉高 SCL */
    delay_us(userconfig_delayus);   /* userconfig_delayus = 1us */
    sIIC_SDA_L;         /* 关键动作：SCL 高时 SDA 下降 → 起始位 */
    delay_us(userconfig_delayus);
    sIIC_SCL_L;
    delay_us(userconfig_delayus);
}

/* 停止信号：SCL 高电平期间，SDA 由低变高 */
static void sIIC_Stop(void)
{
    SDA_OUT();
    sIIC_SCL_L;  sIIC_SDA_L;      /* 先确保 SDA 低 */
    sIIC_SCL_H;                  /* 拉高 SCL */
    delay_us(userconfig_delayus);
    sIIC_SDA_H;                  /* SCL 高时 SDA 上升 → 停止位 */
    delay_us(userconfig_delayus);
}

/* 等待从机应答，带超时（timeout 单位 = userconfig_delayus * timeout） */
static uint8_t sIIC_WaitAck(uint32_t timeout)
{
    uint32_t time = 0;
    SDA_IN();               /* 释放 SDA，让从机拉低产生 ACK */
    sIIC_SDA_H;
    delay_us(userconfig_delayus);
    sIIC_SCL_H;             /* 拉高 SCL，从机在此时把 SDA 拉低 */
    delay_us(userconfig_delayus);

    while (READ_SDA()) {      /* 一直读到 SDA 变低（有 ACK）为止 */
        time++;
        delay_us(userconfig_delayus);
        if (time > timeout) {
```

```
            sIIC_Stop();      /* 超时：补一个停止信号释放总线，避免总线卡死 */
            return 0;
        }
    }
    sIIC_SCL_L;
    SDA_OUT();
    return 1;                /* 收到 ACK */
}

/* 带寄存器地址的读：Start → 写地址 → 写寄存器号 → 重复 Start → 读地址 → 连续读 → Stop
 * 这就是 MPU6050 读 FIFO 时实际走的路径 */
static IIC_Status_t IIC_Mem_Read(uint16_t dev_addr, uint16_t mem_addr,
                                 uint8_t *data, uint16_t size, uint32_t timeout)
{
    sIIC_Start();
    sIIC_SendByte(dev_addr);                /* 写模式（最低位 0） */
    if (!sIIC_WaitAck(timeout)) return IIC_TIMEOUT;

    sIIC_SendByte(mem_addr);                /* 寄存器地址，MPU6050 是 8 位地址 */
    if (!sIIC_WaitAck(timeout)) return IIC_TIMEOUT;

    sIIC_Start();                           /* 重复起始信号（Sr），不插入 Stop */
    sIIC_SendByte(dev_addr | 0x01);          /* 读模式（最低位 1） */
    if (!sIIC_WaitAck(timeout)) return IIC_TIMEOUT;

    for (uint16_t i = 0; i < size; i++)
        /* 最后一个字节回 NACK，告诉从机"不要再给了" */
        data[i] = sIIC_ReadByte(i == (size - 1) ? 0 : 1);

    sIIC_Stop();
    return IIC_OK;
}
```

10.4.2 驱动层：MPU6050.c / MPU6050.h

MPU6050.h 的四个部分。

```
/* ① 姿态数据结构：库作者预留的"分量式"结构，本工程实际未使用 */
typedef struct { float x, y, z; } imu_val;
typedef struct { imu_val gyro, accel; float roll, pitch, yaw; } Imu_t;
```

```
extern Imu_t mpu6050;          /* 定义在 .c 中，但全工程没有使用 */
extern Imu_t RegOri_mpu6050;   /* 同上，预留的"开机零位" */
/* ② 器件地址 */
#define devAddr 0x68    /* AD0 接地 */
/* ③ 寄存器地址 / 位域位置宏 —— 覆盖了 MPU6050 全部 118 个寄存器
 *    命名规则：MPU6050_RA_xxx = Register Address
 *              MPU6050_xxx_BIT / _LENGTH = 该位段在一个字节里的起始位和长度
 *    这些宏是后面 IICwriteBits() 做"读-改-写"的依据 */
#define MPU6050_RA_PWR_MGMT_1       0x6B
#define MPU6050_RA_GYRO_CONFIG      0x1B
#define MPU6050_RA_ACCEL_CONFIG     0x1C
...
/* ④ 对外 API（真正的业务接口只有前 4 个） */
void    MPU6050_initialize(void);   /* 上电初始化：查 ID、设量程、关休眠 */
uint8_t MPU6050_testConnection(void);         /* 读 WHO_AM_I 判断在线 */
void    MPU6050_getMotion6(...);             /*  只有声明，无实现 */
void    MPU6050_getlastMotion6(...);          /*  只有声明，无实现 */
uint8_t MPU6050_getDeviceID(void);
void    MPU6050_InitGyro_Offset(void);        /*  只有声明，无实现 */
void    DMP_Init(void);                      /* DMP 初始化（真正的核心） */
void    Read_DMP(void);                      /* 读一包姿态（10ms 调一次） */
int     Read_Temperature(void);               /* 读温度（本工程未使用） */
```

位域写：整个驱动最容易看不懂的地方。

```
/**************************************************************************
 * 把 data 的 length 位，写到 reg 寄存器的 [bitStart : bitStart-length+1] 位段
 * 参数：addr 7 位器件地址；reg 寄存器地址；bitStart 高位序号；length 位宽；data 待写值
 * 返回：0 = 成功（IIC_OK），非 0 = 失败
 * 原理："读 - 改 - 写"，因为同一个寄存器里还躺着别的配置位

**************************************************************************/
uint8_t IICwriteBits(uint8_t addr, uint8_t reg, uint8_t bitStart, uint8_t length, uint8_t data)
{
    uint8_t b;

    /* ① 先读出寄存器当前值；读失败就直接返回 1（总线异常） */
    if (siic->read_reg(addr << 1, reg, &b, 1, 200) == IIC_OK) {

        /* ② 构造掩码：目标位段清 0，其余位保留 1
         *    0xFF << (bitStart+1)               → 保留位段上方的位
         *    0xFF >> ((8-bitStart)+length-1)     → 保留位段下方的位
```

```
         *
         *    例 1：bitStart=4, length=2（GYRO_CONFIG 的 FS_SEL）
         *         0xFF<<5 = 0xE0；0xFF>>(4+2-1=5) = 0x07 → mask = 0xE7
         *         即"保留 bit7..5 和 bit2..0，清掉 bit4..3"，正确
         *    例 2：bitStart=6, length=1（PWR_MGMT_1 的 SLEEP）
         *         0xFF<<7 = 0x80；0xFF>>(2+1-1=2) = 0x3F → mask = 0xBF
         *         即"清掉 bit6"，正确
         *    例 3：bitStart=7（最高位）
         *         0xFF<<8 = 0xFF00，赋给 uint8_t 后截断成 0x00；
         *         0xFF>>(1+length-1) 保留低位 → 结果只清 bit7，也正确 */
        uint8_t mask = (0xFF << (bitStart + 1)) | (0xFF >> ((8 - bitStart) + length
- 1));

        /* ③ 把 data 的低 length 位"挪"到位段起始处
         *    第一步 <<(8-length)：把有效位顶到最高位（对齐到"最高位为 bit0"的视角）
         *    第二步 >>(7-bitStart)：再右移落到位段的实际位置
         *    例：bitStart=4, length=2, data=0b11
         *        data<<=6 → 0xC0；data>>=3 → 0x18（即 bit4:bit3 = 1:1）  */
        data <<= (8 - length);
        data >>= (7 - bitStart);

        b &= mask;   /* ④ 清掉目标位段 */
        b |= data;   /* ⑤ 写入新值 */

        /* ⑥ 整体写回；返回 IIC_OK(0) 或错误码 */
        return siic->write_reg(addr << 1, reg, &b, 1, 200);
    }
    return 1;        /* 读失败 */
}
/**************************************************************************
 * 单 bit 写：IICwriteBits 的简化版，常用于 SLEEP / I2C_MST_EN / BYPASS_EN
 * 注意：本函数没有检查 read_reg 的返回值，读失败时 b 是未初始化的随机值
**************************************************************************/
uint8_t IICwriteBit(uint8_t dev, uint8_t reg, uint8_t bitNum, uint8_t data)
{
    uint8_t b;
    siic->read_reg(dev << 1, reg, &b, 1, 200);     /* 读原值（未判返回值） */
    b = (data != 0) ? (b | (1 << bitNum))           /* 置位 */
                    : (b & ~(1 << bitNum));         /* 清位 */
    return siic->write_reg(dev << 1, reg, &b, 1, 200);   /* 写回 */
}
```

寄存器配置函数组：

```
/* 时钟源：PWR_MGMT_1 的 CLK_SEL[2:0]
 *   0=内部 8MHz RC  1=X 轴陀螺 PLL  2=Y 轴陀螺 PLL  3=Z 轴陀螺 PLL
 *   4=外部 32.768k  5=外部 19.2M    7=停止时钟
 * 用陀螺 PLL 的原因是：内部 RC 温漂大（±1%），会导致采样节拍不准、积分漂移 */
void MPU6050_setClockSource(uint8_t source)
{
    IICwriteBits(devAddr, MPU6050_RA_PWR_MGMT_1, MPU6050_PWR1_CLKSEL_BIT,  /*
bit2 */
     MPU6050_PWR1_CLKSEL_LENGTH, source);                   /* 3 位 */
}

/* 陀螺量程：GYRO_CONFIG 的 FS_SEL[4:3]
 *  0=±250  1=±500  2=±1000  3=±2000 °/s
 * 本工程用 ±2000，灵敏度 16.4 LSB/(°/s) —— 适合小车急转，代价是分辨率变粗 */
void MPU6050_setFullScaleGyroRange(uint8_t range)
{
    IICwriteBits(devAddr, MPU6050_RA_GYRO_CONFIG, MPU6050_GCONFIG_FS_SEL_BIT,
                 MPU6050_GCONFIG_FS_SEL_LENGTH, range);
}

/* 加速度量程：ACCEL_CONFIG 的 AFS_SEL[4:3]
 *  0=±2g  1=±4g  2=±8g  3=±16g
 * 本工程用 ±2g，灵敏度 16384 LSB/g —— 精度最高，但撞击/跌落时会饱和 */
void MPU6050_setFullScaleAccelRange(uint8_t range)
{
    IICwriteBits(devAddr, MPU6050_RA_ACCEL_CONFIG,
MPU6050_ACONFIG_AFS_SEL_BIT, MPU6050_ACONFIG_AFS_SEL_LENGTH, range);
}

/* 休眠控制：PWR_MGMT_1 的 SLEEP(bit6)，1=睡 0=工作 */
void MPU6050_setSleepEnabled(uint8_t enabled)
{
    IICwriteBit(devAddr, MPU6050_RA_PWR_MGMT_1,
                 MPU6050_PWR1_SLEEP_BIT, enabled);
}

/* 是否让 MPU6050 去当 AUX I2C 的主机（本工程不用磁力计，关掉） */
void MPU6050_setI2CMasterModeEnabled(uint8_t enabled)
{
    IICwriteBit(devAddr, MPU6050_RA_USER_CTRL,
                MPU6050_USERCTRL_I2C_MST_EN_BIT, enabled);
}

/* 是否把主 I2C 与 AUX I2C 直通（接 HMC5883 等从机时才开） */
```

```
void MPU6050_setI2CBypassEnabled(uint8_t enabled)
{
    IICwriteBit(devAddr, MPU6050_RA_INT_PIN_CFG,
                MPU6050_INTCFG_I2C_BYPASS_EN_BIT, enabled);
}
/* 读 WHO_AM_I(0x75)，正常返回 0x68 */
uint8_t MPU6050_getDeviceID(void)
{
    IICreadBytes(devAddr, MPU6050_RA_WHO_AM_I, 1, buffer);  /* buffer 定义的
14 字节全局缓冲 */
    return buffer[0];
}

uint8_t MPU6050_testConnection(void)
{
    return (MPU6050_getDeviceID() == 0x68) ? 1 : 0;
}
```

MPU6050_initialize()：第一段初始化

```
void MPU6050_initialize(void)
{
    /* ① 通信自检：ID != 0x68 就整片复位
     *    ⚠ 这里没有重试、没有延时，传感器没插好 => 不停重启（见 8.3） */
    if (MPU6050_getDeviceID() != 0x68)
        NVIC_SystemReset();

    MPU6050_setClockSource(MPU6050_CLOCK_PLL_YGYRO);      /* CLK_SEL=2，Y 轴
陀螺 PLL */
    MPU6050_setFullScaleGyroRange(MPU6050_GYRO_FS_2000);  /* FS_SEL=3,
±2000°/s */
    MPU6050_setFullScaleAccelRange(MPU6050_ACCEL_FS_2);  /* AFS_SEL=0，±2g */
    MPU6050_setSleepEnabled(0);                          /* 退出休眠 */
    MPU6050_setI2CMasterModeEnabled(0);                  /* 关 AUX 主机模式
*/
    MPU6050_setI2CBypassEnabled(0);                     /* 关直通 */
}
```

注意：这 6 步配置几乎全部被后面的 `DMP_Init()` 覆盖。注释掉也可以。

DMP_Init()：真正的初始化主干。

```
void DMP_Init(void)
{
```

```
    uint8_t temp[1] = {0};

    i2cRead(0x68, 0x75, 1, temp);           /* ① 再确认一次 WHO_AM_I */
    printf("mpu_set_sensor complete ......\r\n");   /*提示信息 */
    if (temp[0] != 0x68) NVIC_SystemReset(); /* ② ID 不对，复位 */

    if (!mpu_init())                    /* ③ 官方初始化：软复位 + 默认参数 */
    {
        /* ④ 打开陀螺+加速度（写 PWR_MGMT_1/2） */
        if (!mpu_set_sensors(INV_XYZ_GYRO | INV_XYZ_ACCEL))
            printf("mpu_set_sensor complete ......\r\n");

        /* ⑤ 配置 FIFO:非 DMP 阶段先把 6 轴原始数据排队,并打开 data-ready 中断 */
        if (!mpu_configure_fifo(INV_XYZ_GYRO | INV_XYZ_ACCEL))
            printf("mpu_configure_fifo complete ......\r\n");

        /* ⑥ 采样率 100Hz：SMPLRT_DIV = 1000/100-1 = 9，
         *    同时自动把 DLPF 设成采样率的一半（→ 落到 42Hz 档） */
        if (!mpu_set_sample_rate(DEFAULT_MPU_HZ))
            printf("mpu_set_sample_rate complete ......\r\n");

        /* ⑦ 把 3062 字节 DMP 固件烧进芯片 DMP 内存（起始 0x0400）
         *    每 16 字节一包，写完立即回读比对，任何一包不一致就失败 */
        if (!dmp_load_motion_driver_firmware())
            printf("dmp_load_motion_driver_firmware complete ......\r\n");

        /* ⑧ 告诉 DMP 传感器的安装方向（3x3 矩阵 → 9bit 标量） */
        if
(!dmp_set_orientation(inv_orientation_matrix_to_scalar(gyro_orientation)))
            printf("dmp_set_orientation complete ......\r\n");

        /* ⑨ 选择 DMP 功能：6 轴低功耗四元数 + 敲击 + 安卓方向 +
         *    原始加速度 + 校准后角速度 + 陀螺自动校准
         *    由此算出每包长度 = 6+6+16+4 = 32 字节 */
        if (!dmp_enable_feature(DMP_FEATURE_6X_LP_QUAT | DMP_FEATURE_TAP |
                DMP_FEATURE_ANDROID_ORIENT | DMP_FEATURE_SEND_RAW_ACCEL |
                DMP_FEATURE_SEND_CAL_GYRO | DMP_FEATURE_GYRO_CAL))
            printf("dmp_enable_feature complete ......\r\n");

        /* ⑩ DMP 输出速率 100Hz：div = 200/100-1 = 1（DMP 内核固定 200Hz） */
        if (!dmp_set_fifo_rate(DEFAULT_MPU_HZ))
            printf("dmp_set_fifo_rate complete ......\r\n");

        run_self_test();                    /* ⑪ 陀螺/加速度自检 + 零偏写入 DMP
```

```
*/

       if (!mpu_set_dmp_state(1))          /* ⑫ 最后一步：启动 DMP 与 DMP 中断
*/
           printf("mpu_set_dmp_state complete ......\r\n");
    }
}
```

姿态读取 Read_DMP()（中断里每 10ms 调一次）：

```
/* 全工程被调用频率最高的 MPU 函数：100Hz，在 EXTI 中断上下文里执行 */
void Read_DMP(void)
{
    unsigned long sensor_timestamp;
    unsigned char more;
    long quat[4];

    /* ① 从 FIFO 取一包（32 字节）并解析成 gyro[3] / accel[3] / quat[4]
     *    返回值未判断：取包失败（FIFO 空/溢出/校验错）时 quat 保持上次内容，
     *    本函数会继续用旧四元数算角度 —— 表现为"姿态偶尔卡住一下" */
    dmp_read_fifo(gyro, accel, quat, &sensor_timestamp, &sensors, &more);

    /* ② 只有当本次包里确实带了四元数，才更新姿态 */
    if (sensors & INV_WXYZ_QUAT)
    {
        /* ③ Q30 定点 → 浮点：除以 2^30 (1073741824) */
        q0 = quat[0] / q30;
        q1 = quat[1] / q30;
        q2 = quat[2] / q30;
        q3 = quat[3] / q30;

        /* ④ 四元数 → 欧拉角（单位：度）；57.3 ≈ 180/π */
        Roll  = asin(-2 * q1 * q3 + 2 * q0 * q2) * 57.3f;
        Pitch = atan2(2 * q2 * q3 + 2 * q0 * q1,
                      -2 * q1 * q1 - 2 * q2 * q2 + 1) * 57.3f;
        Yaw   = atan2(2 * (q1 * q2 + q0 * q3),
                      q0 * q0 + q1 * q1 - q2 * q2 - q3 * q3) * 57.3f;
        /* 注：三个变量 Roll/Pitch/Yaw 是全局非 volatile 变量，
         *     由中断写、主循环读，编译器优化下建议加 volatile（见 8.7） */
    }
}
```

温度读取：

```
/* 官方公式：T(℃) = 36.53 + raw / 340，raw 为有符号 16 位 */
int Read_Temperature(void)
```

```
{
    float Temp;
    Temp = (I2C_ReadOneByte(devAddr, MPU6050_RA_TEMP_OUT_H) << 8)
         +  I2C_ReadOneByte(devAddr, MPU6050_RA_TEMP_OUT_L);

    if (Temp > 32768) Temp -= 65536;      /* 负数补码修正（严格说应判断 >= 32768）
*/
    Temp = (36.53f + Temp / 340) * 10;    /* 放大 10 倍，用整数返回，省一次浮点
传输 */
    return (int)Temp;
}
```

本工程没有调用它。main.c 在 EXTI 回调里用 iic->read_reg(0x68<<1, 0x41, tempbuffer, 2, 200)自己实现了同样的事 (一次读 2 字节，比调用 Read_Temperature() 的两次单字节读快一倍）。

安装方向矩阵：

```
/* 3x3 方向矩阵：芯片坐标系 → 车体坐标系
 *   {-1, 0, 0,
 *     0,-1, 0,
 *     0, 0, 1}
 * 意思是：X、Y 都取反，Z 不变 —— 等价于绕 Z 轴旋转 180°。
 * 物理含义：模块是"倒着"贴在板子上的（芯片 X/Y 朝向与车头方向相反）。
 * 若换板子/换贴装方向，必须改这个矩阵，否则 Roll/Pitch/Yaw 全反。 */
static signed char gyro_orientation[9] = {-1, 0, 0,
                                          0,-1, 0,
                                          0, 0, 1};

/* 把一行方向向量压成 3 bit：
 *   (1,0,0)→0  (-1,0,0)→4  (0,1,0)→1  (0,-1,0)→5  (0,0,1)→2  (0,0,-1)→6
 * 返回值的 bit0~1 = 轴序号(0/1/2)，bit2 = 该轴是否取反 */
static unsigned short inv_row_2_scale(const signed char *row)
{
    if (row[0] > 0) return 0;
    else if (row[0] < 0) return 4;   /* 取反标志 bit2 = 1 */
    else if (row[1] > 0) return 1;
    else if (row[1] < 0) return 5;
    else if (row[2] > 0) return 2;
    else if (row[2] < 0) return 6;
    else return 7;                  /* 全 0，非法行 */
```

```
}

/* 三行各 3 bit，拼成一个 9 bit 的 orient 标量：
 *   bit0~2 = 第 1 行   bit3~5 = 第 2 行   bit6~8 = 第 3 行
 * 本例结果 = 4 | (4<<3) | (2<<6) = 4 + 32 + 128 = 164 = 0x0A4 */
static unsigned short inv_orientation_matrix_to_scalar(const signed char *mtx)
{
    unsigned short scalar;
    scalar  = inv_row_2_scale(mtx);
    scalar |= inv_row_2_scale(mtx + 3) << 3;
    scalar |= inv_row_2_scale(mtx + 6) << 6;
    return scalar;
}
```

自检与零偏：

```
/* 调用官方自检：只有三项全部通过（result == 0x7：陀螺|加速度|磁力计）才把零偏写进
DMP。
 * 本工程没接磁力计，所以 result 基本只会是 0x3，这段"写零偏"实际上很少真正执行。
 * 这也是 Yaw 会缓慢漂移的原因之一。 */
static void run_self_test(void)
{
    int result;
    long gyro_bias[3], accel_bias[3];

    result = mpu_run_self_test(gyro_bias, accel_bias);
    if (result == 0x7) {
        float sens;
        unsigned short accel_sens;

        mpu_get_gyro_sens(&sens);          /* 当前量程下的 LSB/(°/s)，±2000 →
16.4 */
        gyro_bias[0] = (long)(gyro_bias[0] * sens);   /* 归一化偏置 → 原始 LSB
*/
        gyro_bias[1] = (long)(gyro_bias[1] * sens);
        gyro_bias[2] = (long)(gyro_bias[2] * sens);
        dmp_set_gyro_bias(gyro_bias);      /* 写进 DMP 的偏置寄存器 */

        mpu_get_accel_sens(&accel_sens);    /* ±2g → 16384 LSB/g */
        accel_bias[0] *= accel_sens;
        accel_bias[1] *= accel_sens;
        accel_bias[2] *= accel_sens;
        dmp_set_accel_bias(accel_bias);
    }
}
```

10.4.3 通讯层：inv_mpu.c（InvenSense 官方驱动，2885 行）

这一层是芯片原厂代码，本工程只做了 3 处裁剪：平台适配、注释掉 QFN 包（`interrupt_param`）、注释掉 `log` 宏。核心逻辑没动。

10.4.4 固件层：inv_mpu_dmp_motion_driver.c（1367 行，其中 1210 行是数组）

1 ~ 240 行 DMP 内存地址宏（D_0_xx / D_8_xx / D_10_xx …） TAP/方向常量
242 ~ 448 行 ★ static const unsigned char dmp_memory[3062] ← DMP 固件二进制
450 行 static const unsigned short sStartAddress = 0x0400 ← 入口地址
454 ~ 497 行 结构体 dmp_s（回调 + orient + 功能掩码 + fifo_rate + packet_length）
502 ~ 1350 行 dmp_* 系列 API

六、串口通讯

小车的通讯涉及 C10B 与 C50C、C10B 与 K210 之间的串口通讯。C50C 与 C10B 通讯的串口定义如图 6-1；PA2 → USART2_TX（发送），PA3 → USART2_RX（接收），这两个 IO 口是复用 IO，必须软件配置为 AF7 复用模式才变成串口 2。



图 6-1 C50C 串口 2 定义

C10B 与 C50C 通讯的串口定义如图 6-2；PC12 = USART5_TX（串口 5 发送），PD2 = USART5_RX（串口 5 接收）。注意图 6-1 与图 6-2 的发送与接受引脚要交叉连接。

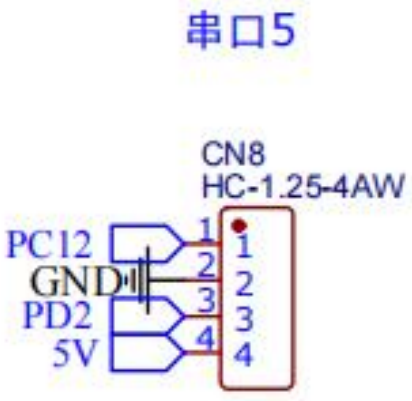
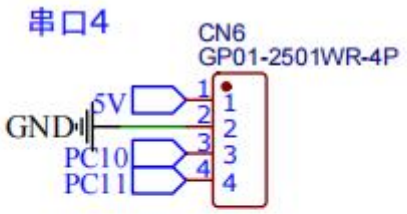


图 6-2 C50C 串口 2 定义

C10B 与 K210 通讯的串口定义如图 6-3；PC10 = UART4_TX,

PC11 = UART4_RX，这个串口不是 USART，无 SCLK、CTS/RTS，只有 TX/RX，不支持同步时钟，仅异步串口。默认就是串口 4，不需要 AFIO 重映射。



OpenMV接口

图 6-3　C10B 与 K210 通讯串口定义

6.1 什么是串口通讯

小车的传感器安装在 C10B 控制板上，如何与主控 C50C 交换数据、K210 推理结果又如何与 C10B 交换呢，这中间必然要有通讯。在众多通信方式里，串口需要很少信号线，布线简单、远距离抗干扰更好、用起来最频繁的一种。

串口通信（Serial Communication）：是一种按位（bit）依次串行传输数据的通信方式，数据在一根线上一位一位排队发送，和并行通信（多条线一次性传多个 bit）相对。日常说的串口大多指 UART/RS232/RS485，嵌入式里最常用 UART。

这里必须纠正一个误解：认为"串行比并行慢"。在同样的位速率下，并行确实一次能搬更多数据；但并行把线数量推上去之后，引脚成本、串扰、各线时序偏移会迅速吃掉它的优势。所以现代高速总线

——USB 3.x、PCIe、千兆以太网、SATA——全部是串行，它们用极高的位速率配合差分信号，把"位"跑得足够快。

6.2 四种电气标准：TTL、RS232、RS485、RS422

串口的逻辑是统一的（低电平表示 0，高电平表示 1，用起始位和停止位界定帧），但"用什么电压表示高和低"是可以换的。这一层换掉，通信能力就完全不同。

表 2-2　四种电气标准对比

标准	电平与传输方式	拓扑与节点数	典型距离	典型场景
TTL（UART 直连）	单端，0 V 表示 0，3.3 V（部分 5 V）表示 1	点对点，两端	板内，几十厘米	开发板调试、同板模块互联
RS232	单端，逻辑 1 为 -3 至 -15 V，逻辑 0 为 +3 至 +15 V	点对点，两端	约 15 m	老式 PC 串口、仪器仪表
RS485	差分，两线间压差 ±1.5 至 6 V	总线型，常见 32 节点，可扩展至上百	约 1200 m（低速）	工业现场、电表、PLC
RS422	差分	点对点全双工，一发多收	约 1200 m	长距离全双工、伺服与编码器

要注意三个关键差别。第一，RS232 的电平是"反"的，而且是高压：逻辑 1 对应负电压，逻辑 0 对应正电压，幅值在 ±3 V 到 ±15 V 之间。所以 TTL 串口和 RS232 串口不能直接对接，中间必须有电平转换芯片（如 MAX3232）。

第二，RS485 是差分传输。它不用一根线对地的电压表示逻辑，而是用两根线之间的压差表示。噪声会同时耦合到两根线上，接收端

取差值时噪声被抵消——这就是它能拉一千多米的原因。代价是 RS485 通常是半双工的，两端不能同时发。

第三，RS422 与 RS485 物理层相近，但拓扑不同。RS422 是点对点全双工（一个发、多个收），RS485 是总线型半双工（多个节点共享总线）。需要长距离又要求全双工时选 RS422。

6.3 传输方向

按数据流动的方向，通信分为三类：单工、半双工、全双工。

表 2-3　三种传输方向

类型	定义	典型例子
单工（Simplex）	数据只能从一端流向另一端，反向没有通路	广播、红外遥控
半双工（Half-duplex）	两端都能发，但同一时刻只能一方占用线路	RS485、对讲机
全双工（Full-duplex）	两端可同时收发，互不干扰	UART 的 TX / RX 两根线、电话

UART 天生是全双工的，因为它有独立的发送线（TX）和接收线（RX）。这一点很重要：全双工意味着你的程序可以在发送数据的同时接收数据，不用考虑"我说的时候你得闭嘴"这种协调问题，调试场景下特别方便。

6.4 同步与异步

同步与异步通讯的区分只有一条：有没有一根独立的时钟线。

6.4.1 同步通讯：靠一根时钟线对齐

同步通讯的核心思想是：发数据的一方同时发一根时钟，收数据的另一方在时钟的边沿去读数据线。因为"什么时候看数据"这件事由时钟线明确规定了，所以数据本身不需要任何起止标记——可以一口气连续发几万个字节，中间不需要停顿，效率极高。代价也很明确：多一根线（至少需要时钟线、数据线、地线三根）；距离受限（时钟线本身也会延迟和畸变，距离一长，时钟和数据到达接收端的先后关系就乱了）；主从速率被绑死（双方必须在同一个时钟节拍下工作，一方慢就整体慢）。

同步通讯的典型应用是 SPI、I2C，以及 USART 的同步模式。

6.4.2 异步通讯：没有时钟线怎么办

异步通讯没有时钟线，双方各自用自己的时钟源产生节拍。既然是各自"掐秒表"，就必须事先约定秒表的快慢——这个约定就是波特率。那怎么知道数据什么时候开始？靠帧结构自同步。

第一步，线路空闲时保持高电平。接收方一直盯着这根线，等待一个下降沿。第二步，发送方要发数据时，先拉低一个位时间，这就是起始位。接收方检测到这个下降沿，立刻把自己的位计时器清零，从这一刻开始按约定的节拍逐位采样。第三步，数据位发完之后，发送方拉高一个位时间作为停止位，保证线路回到高电平，下一帧的下降沿才能被可靠识别。

每传一帧只同步一次。这意味着两件事：帧必须短，因为一帧内只靠一次同步来维持对齐，累积误差正比于帧长，所以 UART 的数据位典型只有 8 位；每帧有固定开销，起始位和停止位是"为了同步付的税"，8N1 下每 10 位只传 8 位有效数据，开销 20%。

6.4.3 同步与异步的对比

表 3-1　同步通讯与异步通讯六维对比

对比维度	同步通讯	异步通讯
时钟线	有独立的时钟线（SCLK / CK）	无，靠双方波特率约定
帧边界	由时钟边沿界定，数据可连续流式传输	由起始位与停止位界定
传输效率	无额外位开销，理论效率接近 100%	8N1 下每 10 位传 8 位有效数据，开销约 20%
速率与距离	速率高但距离短，通常板内	速率中等，借助 RS485 可拉到千米级
硬件成本	至少 3 根线（时钟 + 数据 + 地）	2 根线即可实现全双工（TX、RX）
典型协议	SPI、I2C、USART 同步模式	UART、RS232、RS485

6.5 UART 协议：一帧数据的解剖

6.5.1 一个数据帧有哪些位组成

一帧 UART 数据 (以最常用的 8N1 为例) 从左到右依次由五个字段构成。

表 4-1　UART 一帧的字段构成

字段	长度	电平	作用
空闲位	不定长	高（1）	无数据时的线路状态，接收方在此

字段	长度	电平	作用
			等待起始位
起始位	1 位	低（0）	通知接收方数据要来了，并同步位计时器
数据位	5 至 9 位	视数据而定	真正的有效数据，低位先发
校验位	0 或 1 位	视校验方式而定	可选的奇偶校验位
停止位	1 / 1.5 / 2 位	高（1）	标志本帧结束，保证线路回到空闲态

以发送字节 0x55（二进制 0101 0101）为例，在线上的位序如下表所示。注意 D0 是数据的最低位——0x55 的最低位是 1，所以 D0 等于 1。

表 4-2　发送 0x55 时的线上位序

位置	空闲	起始位	D0	D1	D2	D3	D4	D5	D6	D7	停止位
电平	1	0	1	0	1	0	1	0	1	0	1

6.5.2 波特率：双方约定的同一个节拍

波特率的定义是每秒传输的符号数。在 UART 中一个符号就是一位，所以波特率等于位速率（bit/s）。

表 4-3　常见波特率下的位时间与帧耗时

关系	公式	9600 baud	115200 baud
位时间	1 / 波特率	104.17 μs	8.68 μs
一帧耗时（8N1，10 位）	10 / 波特率	1.0417 ms	86.8 μs

波特率误差为什么必须小？接收方在每一位的中间采样，一位的采样误差不会自己纠正，而是逐位累积。一帧 10 位，如果接收方的实际位时间比发送方快或慢，第 10 位时累积偏差就是 10 倍的单点偏差。要让采样点始终落在位宽的正中间附近，累计偏差不能超过半个位宽——换算下来，双方波特率的相对偏差通常要控制在 2% 以内。

这也是为什么波特率是"标准档位"（9600、19200、38400、57600、115200）而不是任意数值：大家都用同一批档位，才容易对齐。

6.5.3 校验位：奇校验与偶校验

校验位是可选的一位，用来做最基本的错误检测。规则是：让一帧内"1"的总数满足指定的奇偶性。

以 0x55（二进制 0101 0101，其中有 4 个 1，偶数个）为例：无校验时不添加校验位；偶校验要求 1 的总数为偶数，当前已是 4 个，所以校验位取 0；奇校验要求 1 的总数为奇数，所以校验位取 1，使总数变成 5。

表 4-4　0x55（4 个 1）的校验位演算

校验方式	规则	校验位取值	校验后 1 的总数
无校验（None）	不添加校验位	无	4（偶数）
偶校验（Even）	使 1 的总数为偶数	0	4 + 0 = 4，偶数
奇校验（Odd）	使 1 的总数为奇数	1	4 + 1 = 5，奇数

再举一个奇数个 1 的例子：0x57（二进制 0101 0111，其中有 5 个 1，奇数个）。偶校验要补一个 1，使总数变 6；奇校验要补一个 0，总数保持 5。

表 4-5　0x57（5 个 1）的校验位演算

校验方式	规则	校验位取值	校验后 1 的总数
偶校验（Even）	使 1 的总数为偶数	1	5 + 1 = 6，偶数
奇校验（Odd）	使 1 的总数为奇数	0	5 + 0 = 5，奇数

奇偶校验有两个局限。第一，奇偶校验只能检出奇数个位翻转；如果有两位同时翻转，1 的个数的奇偶性不变，校验位检查不出来。第二，奇偶校验只能检错，不能纠错——它告诉你这一帧可能有问题，但不告诉你是哪一位错了，更不会自动改。

还有一个实际影响：校验位占用了位宽。在 STM32 里，字长参数是包含校验位的——如果要传 8 位数据再加 1 位校验，字长要选 9 位。如果要保持帧长 10 位不变，就得把数据位降到 7 位（即 7E1），此时只能表示 ASCII 范围内的字符。

6.5.4 常见帧格式

帧格式通常写成"数据位 + 校验 + 停止位"的形式。如表所示是常用的帧格式。

表 4-6　常见帧格式速查

写法	数据位	校验	停止位	帧总长	适用场合
8N1	8	无	1	10 位	最常用：模块互联、调试打印

写法	数据位	校验	停止位	帧总长	适用场合
8E1	8	偶	1	11 位	需要弱校验的工业设备
8O1	8	奇	1	11 位	约定为奇校验的设备
7E1	7	偶	1	10 位	老式设备，只传 ASCII 字符

参数必须逐项一致。波特率、数据位、校验方式、停止位，四项里有任意一项对不上，收到的就是乱码或者完全收不到。

6.6 STM32 USART 与 HAL 库初始化

协议规定的所有时序细节，其实 STM32 的硬件都替我们实现了。程序要做的，只是把参数填对，然后读写数据寄存器。

6.6.1 USART 与 UART 的区别

STM32 上的串口外设分两种命名。USART（Universal Synchronous / Asynchronous Receiver Transmitter）支持同步和异步两种模式，同步模式下会额外使用一根 CK 引脚输出时钟；UART（Universal Asynchronous Receiver Transmitter）只支持异步模式。

在 STM32F103 系列上，按具体型号不同，通常有 USART1、USART2、USART3，以及更大的封装上带 UART4、UART5。

6.6.2 时钟与引脚

STM32 的每个外设都有一个"使能开关"，不打开它，对这个外设寄存器的任何读写都没有意义。串口的时钟分配如下表。

表 5-1　STM32F103 串口外设的总线归属

外设	挂载总线	总线最高频率（F103 常见配置）
USART1	APB2	72 MHz
USART2 / USART3	APB1	36 MHz
UART4 / UART5	APB1	36 MHz

这解释了一个常见现象：同样配 115200，挂 APB2 的 USART1 最高能跑到 4.5 Mbit/s，而挂 APB1 的 USART2 上限低一半左右——因为波特率是由总线时钟分频得到的，总线时钟越高，能实现的波特率上限越高。忘了开时钟是最常见的错误之一，症状是代码全对但串口毫无反应。

第二件事是引脚复用。USART1 的默认引脚是 TX 等于 PA9、RX 等于 PA10，这是复用功能引脚，不是普通 GPIO。配置时 TX 要设为复用推挽输出，RX 要设为浮空输入或上拉输入。本次小车实训涉及串口通讯的引脚有：

C50C 与 C10B 通讯的串口定义如图 6-1；PA2 → USART2_TX（发送），PA3 → USART2_RX（接收），这两个 IO 口是复用 IO，必须软件配置为 AF7 复用模式才变成串口 2。

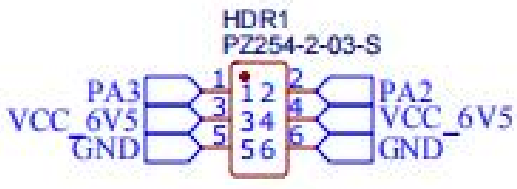


图 6-1 C50C 串口 2 定义

C10B 与 C50C 通讯的串口定义如图 6-2；PC12 = USART5_TX（串口 5 发送），PD2 = USART5_RX（串口 5 接收）。注意图 6-1 与图 6-2 的发送与接受引脚要交叉连接。

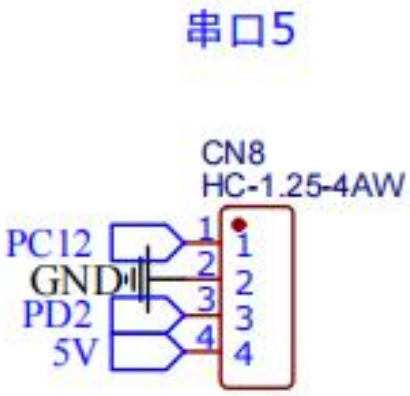


图 6-2　C10B 串口 5 定义

C10B 与 K210 通讯的串口定义如图 6-3；PC10 = UART3_TX, PC11 = UART3_RX，STM32F103 的 USART3 默认引脚是 PB10/PB11，而本板原理图上是 PC10/PC11，属于"部分重映射"（PARTIAL），必须进行引脚重映射，切换 AFIO 内部的复用开关。注意：原理图上标注的串口 4 与程序不一致。



图 6-3　C10B 与 K210 通讯串口定义

数据通路可以这样理解：发送时程序把数据写进数据寄存器（DR），硬件自动补上起始位和停止位，通过发送移位寄存器逐位推到 TX 引脚；接收时 RX 引脚上的电平被采样进接收移位寄存器，凑满约定的位数后放进 DR，并置位 RXNE 标志通知程序来取。也就是说，起止位的添加、位时序的保持、三取二的采样表决，全部由硬件完成。

6.6.3 七个关键参数

在 CubeMX 的 USART1 配置页里，有七个需要确认的参数。逐个说清"改了会怎样"。

表 5-2　CubeMX 串口配置的七个关键参数

参数	常用值	含义与影响
Baud Rate（波特率）	115200	每秒符号数，必须与通信对方完全一致，错一点就是乱码
Word Length（字长）	8 Bits	数据位数。注意含校验位：8 位数据加 1 位校验要选 9 Bits
Parity（校验）	None	无 / 偶 / 奇，与对方一致；本课默认 None
Stop Bits（停止位）	1	1 / 1.5 / 2 位，绝大多数场景用 1
Mode（模式）	Asynchronous	异步 / 同步 / 单线 / 多处理器；本课固定异步
Hardware Flow Control（硬件流控）	Disabled	RTS/CTS 硬件握手，一般禁用；连接不支持流控的设备时必须禁用
NVIC（中断使能）	按需	不用中断就不勾；用中断收发时必须勾上 USART1 global interrupt

这七个参数是收发双方的一份双方合同。你在这边填了什么，对方设备必须填一样的——它决定了线上每一位的高低电平和时间宽度，任何不一致都会变成乱码。

6.6.4 HAL 初始化代码

一般的串口初始化大概有以下六步：

1）开时钟：GPIO 端口在 AHB1；USART1 在 APB2，USART2/3、UART4 在 APB1。

2）配复用：GPIO_PinAFConfig() 把引脚映射到 USARTx 的 AF 编号。

3）配引脚：复用模式 + 推挽 + 50MHz + 上拉。

4）配中断：NVIC 通道、抢占优先级、使能。

5）配串口：8 数据位 / 1 停止位 / 无校验 / 无流控 / 收发模式。

6）开中断 + 使能串口：USART_ITConfig(RXNE) 之后才 USART_Cmd(ENABLE)。

串口初始化函数如下（位于 usart.c），以串口 3 为例。

```
UART_HandleTypeDef huart3;          /* 串口句柄，全局变量，后续所有收发都要用到 */

void MX_USART3_UART_Init(void){
  huart3.Instance = USART3;       /* 外设实例：USART3（挂在 APB1 总线上） */
  huart3.Init.BaudRate = 115200;  /* 与 K210 端 uart 初始化保持一致 */
  huart3.Init.WordLength = UART_WORDLENGTH_8B;      /* 数据位 8 位 */
  huart3.Init.StopBits = UART_STOPBITS_1;          /* 停止位 1 位 */
  huart3.Init.Parity = UART_PARITY_NONE;           /* 无校验（协议自带 BCC） */
  huart3.Init.Mode = UART_MODE_TX_RX;              /* 收为主，TX 暂未使用 */
  huart3.Init.HwFlowCtl = UART_HWCONTROL_NONE;      /* 无硬件流控 */
  huart3.Init.OverSampling = UART_OVERSAMPLING_16;  /* 16 倍过采样 */
  if (HAL_UART_Init(&huart3) != HAL_OK)  /* 失败常见原因：AFIO 时钟没开(重映射
```

```
无效)、DMA 未先初始化 */
  {
    Error_Handler();  }

}
```

最后由 HAL_UART_Init（用句柄里的通信参数配置串口外设 + 按需调用 MSP 完成底层硬件资源初始化）统一落地：它会根据波特率和外设所在总线的时钟频率，算出波特率寄存器的分频值并写入硬件，同时使能外设。

```
void HAL_UART_MspInit(UART_HandleTypeDef* uartHandle)
{
  GPIO_InitTypeDef GPIO_InitStruct = {0};
  if(uartHandle->Instance==UART5)
  {
     //...
  }
  else if(uartHandle->Instance==USART3)
  {
    /* USART3 clock enable */
    __HAL_RCC_USART3_CLK_ENABLE();
    __HAL_RCC_GPIOC_CLK_ENABLE();

    /**USART3 GPIO Configuration
    PC10     ------> USART3_TX
    PC11     ------> USART3_RX
    */
    /* ---------- TX：PC10 → USART3_TX ---------- */
    GPIO_InitStruct.Pin = GPIO_PIN_10;
    GPIO_InitStruct.Mode = GPIO_MODE_AF_PP;
    GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_HIGH;
    HAL_GPIO_Init(GPIOC, &GPIO_InitStruct);

    /* ---------- RX：PC11 → USART3_RX ----------
       这里是本工程最关键的一根接收线：K210 的 TX 接过来。
       浮空输入 + 杜邦线较长时容易出现误码，建议硬件上串 22~100Ω 电阻并缩短走线。*/
    GPIO_InitStruct.Pin = GPIO_PIN_11;
    GPIO_InitStruct.Mode = GPIO_MODE_INPUT;
    GPIO_InitStruct.Pull = GPIO_NOPULL;
    HAL_GPIO_Init(GPIOC, &GPIO_InitStruct);
```

```
    /* ---------- 引脚重映射 ----------
       STM32F103 的 USART3 默认引脚是 PB10/PB11，而本板走的是 PC10/PC11，
       属于"部分重映射"（PARTIAL），必须调用这句切换 AFIO 内部的复用开关。
     */
    __HAL_AFIO_REMAP_USART3_PARTIAL();

    /* ==================== USART3 接收 DMA 配置 ====================
       为什么必须是 DMA1_Channel3？
       STM32F1 的 DMA 请求映射是硬连线的，查参考手册的 DMA1 请求映射表：
       Channel2 → USART3_TX
       Channel3 → USART3_RX   ← 本工程用的就是这个
       通道号写错的话，DMA 永远等不到请求，串口接收会一直超时。
    ================================================================*/
    hdma_usart3_rx.Instance = DMA1_Channel3;
    hdma_usart3_rx.Init.Direction = DMA_PERIPH_TO_MEMORY;    /* 传输方向：外设
→ 内存（接收） */
    hdma_usart3_rx.Init.PeriphInc = DMA_PINC_DISABLE;       /* 外设地址不自增：
源永远是 USART3->DR 同一个地址 */
    hdma_usart3_rx.Init.MemInc = DMA_MINC_ENABLE;           /* 内存地址自增：逐
字节填进 k210_buffer[] 的连续位置 */
    hdma_usart3_rx.Init.PeriphDataAlignment = DMA_PDATAALIGN_BYTE;   /* 外设
侧数据宽度：8bit（串口一次就是一个字节） */
    hdma_usart3_rx.Init.MemDataAlignment = DMA_MDATAALIGN_BYTE;      /* 内存
侧数据宽度：8bit */
    hdma_usart3_rx.Init.Mode = DMA_NORMAL;     /* 单次(NORMAL)：传满长度就停，
需在回调里重新启动。 若改成 CIRCULAR 循环模式，缓冲区会被从头覆盖，必须配合半传输/
双缓冲使用，否则正在解析的数据会被冲掉 */
    hdma_usart3_rx.Init.Priority = DMA_PRIORITY_LOW;    /* 通道仲裁优先级：多个
通道抢总线时用，与 NVIC 优先级是两回事 */
    if (HAL_DMA_Init(&hdma_usart3_rx) != HAL_OK)
    {
      Error_Handler();
    }

    /* 把 DMA 句柄挂到 UART 句柄上（写 hdmarx 指针）。之后 HAL_UART_Transmit_DMA
/ ReceiveToIdle_DMA 才能顺着 huart->hdmarx 找到通道 */
    __HAL_LINKDMA(uartHandle,hdmarx,hdma_usart3_rx);

    /* USART3 中断：优先级 5，主要用途是捕获 IDLE（总线空闲）标志，用来判断"这一帧
数据发完了"，从而触发 HAL_UARTEx_RxEventCallback()。 */
    HAL_NVIC_SetPriority(USART3_IRQn, 5, 0);
    HAL_NVIC_EnableIRQ(USART3_IRQn);
  }
}
```

6.6.5 调用链：谁调用了它

初始化函数本身不会自动执行，它在启动流程中的位置如下。顺序不能颠倒：SystemClock_Config 必须在前，因为串口波特率的计算依赖系统时钟；MX_GPIO_Init 也必须在串口初始化之前，否则引脚还没配成复用功能。

```
int main(void){
  HAL_Init();               /* 1. 初始化 HAL 库、配置 SysTick */
  SystemClock_Config();     /* 2. 配置系统时钟（决定 APB1 / APB2 频率） */
  MX_GPIO_Init();
  MX_DMA_Init();            /* 开 DMA1 时钟 + 配置 DMA1_Channel3 中断 */
  MX_UART5_Init();          /* UART5  -> C50C 底盘，只发 */
  MX_USART3_UART_Init();    /* USART3 -> K210 视觉，DMA + IDLE 收 */
  //启动 DMA 接收 K210 数据
  HAL_UARTEx_ReceiveToIdle_DMA(&huart3,k210_buffer,30);
  while (1)  {
    /* 主循环：在这里做收发 */
  }
}
```

6.7 三种收发方式与数据校验

同一个 HAL_UART_ 前缀，背后是三种完全不同量级的用法。阻塞是 CPU 干等，中断是 CPU 被打断一次，DMA 是别人替 CPU 搬。

6.7.1 阻塞式收发：最简单的用法

HAL 库提供了最直接的同步收发函数，用法如下。

```
uint8_t tx_buf[] = "Hello, UART!\r\n";    /* 14 个字节，含 \r\n */
uint8_t rx_byte;
/* 阻塞发送：把 14 个字节全部发完才返回，超时 100 ms */
HAL_UART_Transmit(&huart1, tx_buf, 14, 100);
```

```
/* 阻塞接收：等到收到 1 个字节，或等待超过 100 ms 后超时返回 */
HAL_UART_Receive(&huart1, &rx_byte, 1, 100);
```

关于 timeout 参数，单位是毫秒，是"最多愿意等多久"，不是"必须等多久"；还有一个特殊值 HAL_MAX_DELAY，表示无限等待，如果写了它而对方又不发数据，程序就永远停在那里。

阻塞式的问题在哪？发送阻塞尚可接受，因为耗时是可预估的——14 个字节在 115200 baud 下约需 1.2 ms，这个时间通常在容忍范围内。接收阻塞则非常危险：如果对方不发数据，程序会一直卡在这一行，主循环里其他所有逻辑、按键扫描、状态更新全部停摆。在带看门狗的系统里，这会直接导致复位。

适用场景：上电时打印一行版本信息、调试期打印变量值、发送频率很低的短报文。

6.7.2 中断式收发：不让 CPU 空等

中断的思路是：把"等待"这件事交给硬件，数据到了再打断 CPU。典型用法是先挂一个接收请求，然后主循环该干什么干什么。

```
uint8_t rx_byte;
volatile uint8_t rx_flag = 0;    /* 用 volatile，因为会被中断修改 */
/* 在主循环开始前，先开启一次中断接收 */
HAL_UART_Receive_IT(&huart1, &rx_byte, 1);
```

接收完成时，HAL 库会自动调用用户实现的回调函数：

```
/* 每接收完指定长度的数据，HAL 库会调用这个回调 */
void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart){
```

```
  if (huart->Instance == USART1)  {
    rx_flag = 1;          /* 只做标记，不做耗时操作 */
    /* 关键：重新开启中断接收，否则只能收到第一个字节 */
    HAL_UART_Receive_IT(&huart1, &rx_byte, 1);
  }
}
```

整个链路是这样走的：数据到达 RX 引脚，接收移位寄存器凑满一帧；硬件把数据放进 DR，并置位 RXNE（Receive Data Register Not Empty）标志；NVIC 检测到中断请求，打断 CPU 当前执行流，跳转到中断服务函数；HAL 库的中断处理函数 HAL_UART_IRQHandler 读走 DR 里的数据；如果这次接收凑满了请求的长度，HAL 库调用 HAL_UART_RxCpltCallback；你的回调函数处理数据，并重新调用 HAL_UART_Receive_IT 准备接收下一个字节。

这里有三个必须强调的要点。第一，回调里必须重新开启接收。HAL_UART_Receive_IT 是一次性的——它只负责接收你指定长度的数据，收满就停。如果回调里不再调一次，结果就是"只能收到第一个字节"。

第二，接收长度决定回调触发时机。上面写成 1，就是每收 1 个字节触发一次回调。如果要收一整个报文，要么把长度写成报文的固定长度，要么用后面的 DMA 加空闲中断方案。

第三，中断里只做搬运，不做耗时的活。不要在回调里调用 printf、不要延时、不要做复杂的字符串解析。中断服务时间越长，越容易错过下一个字节。发送侧对称地有 HAL_UART_Transmit_IT 和 HAL_UART_TxCpltCallback，用法一致。

6.7.3 DMA 收发：把搬运工作交给专用硬件

DMA 是 Direct Memory Access（直接存储器访问）的缩写。它的核心思想是：在两个存储区域之间搬数据这件事，不需要 CPU 参与。在串口发送的场景里，数据要从 SRAM 里的缓冲区搬到 USART 的数据寄存器。用阻塞方式，这个搬运由 CPU 一条条指令完成；用 DMA，只要设置好源地址、目标地址和搬运长度，DMA 控制器就自己把活干掉。

```
uint8_t tx_buf[] = "Hello, DMA!\r\n";
uint8_t rx_buf[64];
/* DMA 发送：函数立刻返回，后续搬运由 DMA 控制器完成 */
HAL_UART_Transmit_DMA(&huart1, tx_buf, sizeof(tx_buf) - 1);
/* DMA 接收（定长）：收满 64 字节后触发回调 */
HAL_UART_Receive_DMA(&huart1, rx_buf, 64);
```

不定长数据的接收，是 DMA 真正体现价值的地方。串口收到的报文长度往往不固定，用定长 DMA 不知道设置多长合适。解决办法是把空闲中断（IDLE）和 DMA 结合起来：DMA 负责持续把收到的字节搬进缓冲区，一字节一中断的负担省掉了；空闲中断负责判断这一包什么时候结束——当总线在超过一个位时间后仍然没有新

数据，硬件就认为总线空闲了，触发 IDLE 中断。HAL 库对应的是 HAL_UARTEx_ReceiveToIdle_DMA。

DMA 的一个坑：DMA 缓冲区是异步读写的。DMA 在后台往缓冲区里写数据的同时，你的程序可能正在读同一块缓冲区。如果处理速度跟不上接收速度，上一包数据还没处理完，下一包就覆盖上来了。稳妥的做法是收到一包后先拷贝出来处理，或者用双缓冲。

6.7.4 三种方式对比与选型

三种收发方式的多维度对比见表 xxx。

表 6-1 阻塞、中断、DMA 三种收发方式对比

对比维度	阻塞式	中断式	DMA
CPU 占用	全程被占用，纯空等	每字节一次中断，开销小	几乎为零，搬运由硬件完成
实时性	差，会阻塞主循环	好，主循环不受影响	最好
适用场景	上电日志、低频调试打印	命令交互、逐字节解析的协议	批量数据、不定长收包、高吞吐
实现难度	低	中，需处理回调与重入	中高，需配置 DMA 通道与中断
典型 API	HAL_UART_Transmit / Receive	HAL_UART_Transmit_IT / Receive_IT	HAL_UART_Transmit_DMA / ReceiveToIdle_DMA

选型原则是：没有最好，只有最合适。先看数据量和实时性要求，再看实现成本。只在启动时打印一行日志，阻塞式足够；要随时响应上位机发来的单字符命令，用中断式；要接收不定长的数据包、同时主循环还要跑别的任务，用 DMA 加空闲中断。

6.7.5 数据校验：奇偶位之外还有两道防线

前面讲的奇偶校验位，只是最基础的位级校验。实际工程中的数据校验是分三层的。

表 6-2　数据校验的三个层级

层级	手段	能对付什么错误	局限
第一层：位级	奇偶校验位（Parity）	单个位翻转	只能检出奇数个位错误，不能纠错
第二层：帧级	校验和（累加和 / 异或和）	整帧中若干字节出错	实现简单，对抗突发错误能力弱
第三层：帧级	CRC16 / CRC32	连片突发错误	计算稍复杂，但检错能力最强

第一层奇偶校验位由 UART 硬件自动生成和检查，不需要写代码，优点是零成本，缺点是只能管一位错。第二层校验和把整帧数据的各个字节累加起来（取低 8 位），或者逐字节异或，把结果附加在帧尾一起发送，接收方重新算一遍再比对，实现只要几行代码，但两个字节同时出错可能互相抵消，检错能力有限。第三层 CRC（循环冗余校验）通过多项式除法产生校验值，对连续多位同时出错这种情况的检出能力远强于简单求和，以太网、Flash 存储、无线通信协议基本都用 CRC。

结论：奇偶校验位只提高了一位错的检出概率；真正保证数据完整性的是帧级校验。三层可以叠加使用——物理层管单个位的噪声，帧级 CRC 管整包数据的完整性。

6.8 排错与调试实战

串口调不通的原因高度集中，也就那么几种，硬件原因有电平不匹配、发送和接受没有交叉；软件的原因通常是收发两边的参数设置不一致。

6.8.1 串口通讯故障排查

下表按"现象 → 原因 → 排查动作"整理，可以直接对照排查。

表 7-1　串口通讯常见故障排查表

现象	最常见原因	排查动作
收到的全是乱码	两边波特率不一致；系统时钟配置错误	逐项核对波特率；用逻辑分析仪量出一个位的实际宽度，反推真实波特率
只能收到第一个字节	中断模式下没有在回调里重新开启接收	检查 HAL_UART_RxCpltCallback 里是否再次调用了 HAL_UART_Receive_IT
完全没有反应	TX/RX 没有交叉连接；没有共地；串口被其他程序占用	确认本方 TX 接对方 RX、GND 对接；关闭占用同一串口的烧写工具或串口助手
偶发丢数据	没有接收缓冲，新数据覆盖旧数据；中断优先级太低	引入环形缓冲区，或改用 DMA 加空闲中断接收
一上电就收到一个乱码字符	引脚上电瞬间电平抖动被误判为起始位	忽略开机后收到的第一个字节，或加超时与校验后丢弃
收到的数据全是 0 或全是 0xFF	电平标准不匹配，TTL 直连了 RS232 电平	检查是否需要电平转换芯片（如 MAX3232）

总结：乱码先查波特率，不通先查线序和共地，这两个原因占了绝大多数；中断收不到第二个字节，几乎总是忘了重新开启接收；接线时 TX 接对方的 RX，这是最容易接错的地方，因为直觉上会觉得"同名接同名"。

6.8.2 串口调试

串口调试的时候可以借助 USB 转串口模块和串口调试助手来观察收发的数据包是否和我们预想的一致，帮助排查通讯故障。串口调试的硬件连接如图 6-8-1，注意 RX 、TX 要交叉连接，USB 转串口模块的电平设为 3.3V。连接线可以使用两头是公头的杜邦线，一头用螺丝刀拧在 USB 转串口的接线端子上，另一头插在原先的通讯线孔里。

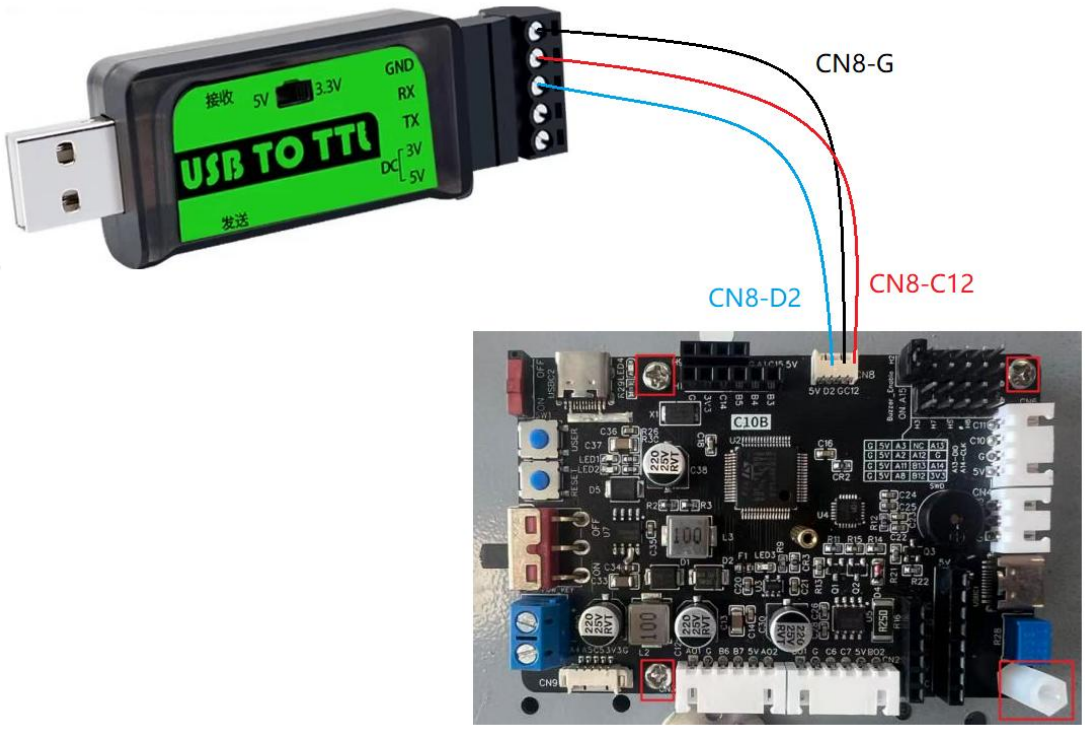


图 6-8-1 C10B 串口 5 调试接线图

串口调试助手的设置如图 6-8-2，串口配置部分要与单片机内的参数一致，其中端口号根据自己电脑设备管理器->端口下的实际值设置，如图 6-8-3，线连好后“打开串口”即可在接收窗口内看到实际数据，如果是乱码或者没有数据，可以按照 6.8.1 节进行故障排查。



图 6-8-2 串口调试助手设置



图 6-8-3 串口端口号查询

6.9 程序解读

6.9.1 C10B 串口部分代码

C10B 控制板使用了三路串口，参数都是 115200-8-N-1、无流控、16 倍过采样，串口 1 用来打印调试信息，串口 3 用来接受 K210 的信息，串口 5 用来向 C50C 发送信息，协议采用"帧头 + 定长 + BCC 异或校验 + 帧尾"的结构。

初始化前面已经介绍，下面主要对串口 3 的接受和串口 5 的发送进行分析。

串口 3 接收由 K210 串口 1 发送来的数据包，采用 DMA+IDLE 方式接收，协议详细格式如表 6-9-1：

表 6-9-1 串口 3 通讯协议

偏移	字段	类型	说明
0	Head	u8	固定 0xCC
1~2	Cam_W	u16	图像宽度（如 320）
3~4	Cam_H	u16	图像高度（如 240）
5~6	cx	u16	目标中心 X（像素）
7~8	cy	u16	目标中心 Y（像素）
9~10	size	u16	目标面积（像素数）
11	BCCcheck	u8	字节 0~10 逐字节异或
12	End	u8	固定 0xDD

示例：图像大小 320×240、目标中心 (160,120)、面积 100

数据帧：CC 40 01 F0 00 A0 00 78 00 64 00 C1 DD

校验过程：CC^40^01^F0^00^A0^00^78^00^64^00 = 0xC1，与第 11 字节相等即通过。

```
/*=========================================================================
 *  串口 DMA 接收事件回调 —— K210 数据处理的核心入口
 * =========================================================================*/
void HAL_UARTEx_RxEventCallback(UART_HandleTypeDef *huart, uint16_t Size)
{
    if( huart == &huart3 && HAL_UART_STATE_READY == huart->RxState )
    {
        /* 收到数据即刷新"视觉在线"状态：
           清零计数（1 秒超时的计时器），并置位在线标志。
           这两个变量在 HAL_GPIO_EXTI_Callback() 里被读取/累加，
           跨中断共享但都是单字节访问，实际风险很低。 */
        k210_active_cnt=0;
        k210_active_flag=1;

        /* 把本次 DMA 收到的 Size 个字节逐字节喂给组帧状态机。
           注意这里必须循环全部字节，不能只取前 13 个：
           K210 连续发送时一次可能收到 26 字节（两帧），
           少喂就会丢帧；多喂也无害，状态机会自己找帧头。
           返回值（是否刚好凑成一帧）本工程不使用。 */
        for(uint8_t i=0;i<Size;i++)
        {
            k210_data_callback(k210_buffer[i]);    //K210.C
        }

        /* 重新启动 DMA + IDLE，准备接下一批数据。
           因为 DMA 配置为 NORMAL 单次模式，传完/空闲后通道就停了，
           必须每次重新启动，才能形成持续接收。
         */
        HAL_UARTEx_ReceiveToIdle_DMA(&huart3,k210_buffer,30);
    }
}
```

```
/**
  * 功能：逐字节组帧，每收到 1 个字节调用一次
  * 参数：recv 本次收到的字节
  * 返回值：1 = 刚刚成功解析出一帧完整数据；0 = 还在攒帧或校验失败
```

```
 * 说明：本函数由 main.c 的 HAL_UARTEx_RxEventCallback() 在中断上下文调用，
 *         所以内部绝对不能有阻塞操作，也不要调用 printf。
 *         返回值目前主循环未使用（数据已直接写入全局 k210），
 *         保留它是为了将来改成事件/回调式使用时不必修改接口。
 */
uint8_t k210_data_callback(uint8_t recv)
{
    static uint8_t recvlen = sizeof(k210);      /* 一帧应有字节数 = 13，随结构体自动变化 */
    static uint8_t recv_data[sizeof(k210)];     /* 攒帧临时缓冲；static 保证跨调用保持内容 */
    static uint8_t recv_counts = 0;             /* 已攒字节数，同时充当写指针下标 */

    uint8_t data_ready = 0;                     /* 本次调用是否凑齐并校验通过一帧 */
```

```
    /* ---------- 第 1 步：先写入 ----------
       先写入再判断，由于 recv_counts 一旦等于 13 就会在下面被清零，此处最大写索引恰为 12，
       正好落在 recv_data[0..12] 范围内，不越界（属于"擦边安全"）。
       注意：如果以后把 recvlen 改大、或结构体增加字段，必须改成
       "先判边界再写"，否则会写穿数组、踩坏相邻的静态变量。 */
    recv_data[recv_counts] = recv;
```

```
    /* ---------- 第 2 步：帧同步（决定计数怎么走） ----------
       只有两种情况下才继续计数：
         1) 收到帧头 0xCC        -> 说明新的一帧开始
         2) recv_counts > 0      -> 说明已经在帧内，继续往后收
       其余情况（空闲态收到非帧头字节）直接丢弃，保持等待帧头。 */
    if (recv == K210_HEAD || recv_counts > 0)
        recv_counts++;
    else
        recv_counts = 0;
```

```
    /* ---------- 第 3 步：收满 13 字节，做合法性检查 ---------- */
    if (recv_counts == recvlen)
    {
        recv_counts = 0;                        /* 无论成功失败都先清零，为下一帧做准备 */
```

```
        if (recv_data[recvlen - 1] == K210_END) /* 检查 1：末字节必须是帧尾 0xDD */
        {
```

```
            /* 检查 2：BCC 校验。
               参与校验的是下标 0~10（Head .. size）共 11 字节，
               待比对的校验值在下标 11，帧尾在下标 12。
               因此校验长度与校验值下标都是 recvlen-2 = 11，正好对应。 */
            if (recv_data[recvlen - 2] == calculateBCC(recv_data, recvlen - 2))
            {
                /* 两项检查都通过 -> 把临时缓冲整体搬到全局结构体。  */
                uint8_t *recvptr = (uint8_t *)&k210;
                for (uint8_t i = 0; i < recvlen; i++)
                {
                    *recvptr = recv_data[i];
                    recvptr++;
                }
                /* 走到这里，k210 里已经是一帧完整可信的数据，
                   主循环可以安全读取（cx/cy/size 供显示与上报）。 */
                data_ready = 1;
            }
            /* 校验失败：静默丢弃这一帧，等下一个帧头。
               不做重传/请求，因为视觉数据流是持续刷新的，
               丢一帧对控制几乎没有影响，重传反而增加复杂度。 */
        }
    }
```

```
    return data_ready;
}
```

串口 5 将从 K210 接收来的数据加上温度以及超声波测得的距离一起打包发送给 C50C，协议详细格式如表 6-9-2：

表 6-9-2 串口 5 通讯协议

偏移	字段	类型	说明
0	Head1	u8	0xCC
1	Head2	u8	0xDD
2	k210_alive	u8	1=1 秒内收到过 K210 数据；0=视觉掉线
3~4	k210_cx	u16	目标中心 X
5~6	k210_size	u16	目标面积
7~8	distance	u16	距离，单位 mm
9~10	temperature	s16	温度 ×100（26.50℃ → 2650）
11	BCCcheck	u8	字节 0~10 异或

发送示例：alive=1、cx=160、size=100、距离 350mm、温度 26.50℃

数据帧：CC DD 01 A0 00 64 00 5E 01 5A 0A DB

```
/* ---------------- 20Hz 向 C50C 上报一帧（走 UART5） ----------------
          触发节拍：MPU6050 中断 100Hz，每 5 次发一帧 -> 20Hz。
          帧格式（共 12 字节，定义见 main.h 的 C10B_Sendmsg）：
            [0]    Head1        0xCC  帧头 1
            [1]    Head2        0xDD  帧头 2
            [2]    k210_alive   1=1 秒内收到过 K210 数据，0=视觉掉线
            [3~4]  k210_cx      目标中心 X（小端）
            [5~6]  k210_size    目标面积（小端）
            [7~8]  distance     距离，单位 mm
            [9~10] temperature  温度 x100，单位 0.01 摄氏度
            [11]   BCCcheck     前 12 字节异或校验
          为什么借这个中断发：
            复用 MPU6050 的 100Hz 节拍做定时，省一个硬件定时器；
            12 字节 @115200 约耗时 1.04ms，相对 10ms 中断周期有余量。
          风险提示：
            HAL_UART_Transmit 是阻塞式，这里超时给了 500ms。
            若 C50C 未接/线路断开导致 TXE 迟迟不来，中断会被卡住最长 0.5s，
            期间 MPU6050 中断全部丢失（姿态角停更、超声波停发）。
            调试时可以把超时降到 5~10ms 看，或改用 HAL_UART_Transmit_DMA 发送。
*/
        c50c_send_cnt++;
        if( c50c_send_cnt>=5 )
        {
            c50c_send_cnt = 0;
            c10b_send.Head1 = C10B_HEAD1;          /* 帧头 1 0xCC */
            c10b_send.Head2 = C10B_HEAD2;          /* 帧头 2 0xDD */
            c10b_send.k210_alive = k210_active_flag; /* 视觉在线标志（1s 超时判定） */
            c10b_send.k210_cx = k210.cx;   /* 目标中心 X，直接来自 K210 帧 */
            c10b_send.k210_size = k210.size;       /* 目标面积 */
            c10b_send.distance = ranger_dis*1000;  /* m -> mm；*/
            c10b_send.temperature = temperature*100; /* 摄氏度 x100，-40~85℃对应 -4000~8500 不溢出 */
            uint8_t* sendptr = (uint8_t*)&c10b_send; /* 按字节视图访问结构体，用于算校验和发送 */
            c10b_send.BCCcheck = calculateBCC(sendptr,sizeof(c10b_send)-1);
/* 前 11 字节异或 */
```

```
            HAL_UART_Transmit(&huart5,sendptr,sizeof(c10b_send),500);
/* 阻塞发送 12 字节 ,超时 500ms*/
        }
```

6.9.3 C50C 串口部分代码

C50C 控制板使用了四个串口，我们主要关注与 C10B 通讯的串口 2，其通讯协议与表 6-9-2 一致。

所有串口初始化都在 HARDWARE/uartx.c 文件中。

```
/**
  * 功能：USART2 初始化（PA2=TX / PA3=RX）
  * 参数：bound 波特率，本工程传 115200
  * 说明：专门对接 K210 视觉模块，接收 C10B 格式的巡线/测距数据。
  *       本函数没有用 uartx.h 里的宏，引脚是写死的，改写时注意。
  */
void USART2_Init(u32 bound)
{
    GPIO_InitTypeDef GPIO_InitStructure;
    USART_InitTypeDef USART_InitStructure;
    NVIC_InitTypeDef NVIC_InitStructure;

    //对应引脚、对应串口时钟开启
    RCC_AHB1PeriphClockCmd(RCC_AHB1Periph_GPIOA,ENABLE);
    RCC_APB1PeriphClockCmd(RCC_APB1Periph_USART2,ENABLE);   // USART2 在
APB1 上
```

```
    //AF 初始化
    GPIO_PinAFConfig(GPIOA,GPIO_PinSource2,GPIO_AF_USART2);
    GPIO_PinAFConfig(GPIOA,GPIO_PinSource3,GPIO_AF_USART2);
```

```
    //对应引脚端口初始化
    GPIO_InitStructure.GPIO_Pin = GPIO_Pin_2;   // PA2 = TX
    GPIO_InitStructure.GPIO_Mode=GPIO_Mode_AF;
    GPIO_InitStructure.GPIO_OType=GPIO_OType_PP;
    GPIO_InitStructure.GPIO_Speed=GPIO_Speed_50MHz;
    GPIO_InitStructure.GPIO_PuPd=GPIO_PuPd_UP;
    GPIO_Init(GPIOA, &GPIO_InitStructure);
```

```
    GPIO_InitStructure.GPIO_Pin = GPIO_Pin_3;   // PA3 = RX
    GPIO_InitStructure.GPIO_Mode=GPIO_Mode_AF;
    GPIO_InitStructure.GPIO_OType=GPIO_OType_PP;
    GPIO_InitStructure.GPIO_Speed=GPIO_Speed_50MHz;
    GPIO_InitStructure.GPIO_PuPd=GPIO_PuPd_UP;
    GPIO_Init(GPIOA, &GPIO_InitStructure);
```

```
    //UsartNVIC 配置
    NVIC_InitStructure.NVIC_IRQChannel = USART2_IRQn;
    //抢占优先级
    NVIC_InitStructure.NVIC_IRQChannelPreemptionPriority= 3;
    //子优先级
    NVIC_InitStructure.NVIC_IRQChannelSubPriority = 0;
    //IRQ 通道使能
    NVIC_InitStructure.NVIC_IRQChannelCmd = ENABLE;
    //根据指定的参数初始化 NVIC 寄存器
    NVIC_Init(&NVIC_InitStructure);
```

```
    //USART 初始化设置
    USART_InitStructure.USART_BaudRate = bound; //串口波特率
    USART_InitStructure.USART_WordLength = USART_WordLength_8b; //字长为 8 位
数据格式
    USART_InitStructure.USART_StopBits = USART_StopBits_1; //一个停止位
    USART_InitStructure.USART_Parity = USART_Parity_No; //无奇偶校验位
    USART_InitStructure.USART_HardwareFlowControl =
USART_HardwareFlowControl_None; //无硬件数据流控制
    USART_InitStructure.USART_Mode = USART_Mode_Rx | USART_Mode_Tx; //收发模
式
    USART_Init(USART2, &USART_InitStructure);      //初始化串口 2

    USART_ITConfig(USART2, USART_IT_RXNE, ENABLE); //开启串口接受中断

    USART_Cmd(USART2, ENABLE);                     //使能串口 2
}
```

BALANCE/uartx_callback.c 文件包含串口中断服务与协议解析实现，是整车的"串口通信大脑"。

```
//============================================================================
//                          K210 视觉模块通信（USART2）
//============================================================================

// 解析完成后的 K210 数据（供 balance_task / show_task 使用）
C10B_Recvmsg c10b_recv={0};
```

```
// 一帧的固定长度，由结构体大小决定（#pragma pack(1) 保证不带填充，共 12 字节）
const uint8_t c10b_datalen = sizeof(C10B_Recvmsg);
// 接收过程中的原始字节缓存
uint8_t c10b_buffer[c10b_datalen];
// 通信超时计数：每进一次 balance_task 就加 1，收到合法帧就清零
uint32_t c10b_active_cnt = 0;
// 数据有效标志：1 表示最近收到过合法帧（未被判定为掉线）
uint8_t c10b_active_flag=0;
volatile uint8_t check1=0;
volatile uint8_t check2=0;
```

```
/**
  * 功能:   USART2 中断服务函数：接收 K210 视觉模块数据（C10B 协议）
  * 说明:   协议格式（12 字节，小端序，见 uartx_callback.h 的 C10B_Recvmsg）：
  *         [0]0xCC [1]0xDD   双字节帧头
  *         [2]     k210_alive   K210 存活标志
  *         [3~4]   k210_cx      目标中心 X 坐标（uint16）
  *         [5~6]   k210_size    目标框大小      （uint16）
  *         [7~8]   distance     测距值          （uint16，mm）
  *         [9~10]  temperature  温度            （short）
  *         [11]    BCC 校验（前 11 字节异或）
  *
  *         实现方式：两状态状态机
  *           sm 为 0 表示找帧头（要求连续两字节 0xCC、0xDD）
  *           sm 为 1 表示顺序填满缓冲区，长度到了再校验
  */
void  USART2_IRQHandler(void)
{
    u8 Usart_Receive;
    static uint8_t Last_Recv;   // 上一个字节，用于识别双字节帧头
    static uint8_t recv_cnt=0;  // 当前帧已收字节数
    static uint8_t sm = 0;      // 状态机状态：0-找帧头 1-收数据

    if(USART_GetITStatus(USART2, USART_IT_RXNE) != RESET) //判断是否接收到数据
    {
        Usart_Receive = USART_ReceiveData(USART2);//Read the data //读取数据
        // 读 DR 即自动清除 RXNE 标志，无需手动清
```

```
        switch( sm )
        {
            case 0:
                // 要求"上一字节是 0xCC、本字节是 0xDD"才认为帧头成立，
                // 这样能滤掉数据区里偶然出现的单个 0xCC
                if( Usart_Receive==C10B_HEAD2 && Last_Recv==C10B_HEAD1 )
```

```
                {
                    sm=1;
                    c10b_buffer[0]=C10B_HEAD1;
                    c10b_buffer[1]=C10B_HEAD2;
                    recv_cnt=2;
                }
                break;
            case 1:
                // 顺序填充缓冲区
                c10b_buffer[recv_cnt++] = Usart_Receive;
                if( recv_cnt == c10b_datalen )   // 整帧收齐
                {
                    sm=0;   // 回到找帧头状态，准备下一帧
                    // 校验最后一字节是否为前面数据的 BCC
                    if( c10b_buffer[c10b_datalen-1] ==
calculateBCC(c10b_buffer,c10b_datalen-1) )
                    {
                        c10b_active_cnt=0;      // 收到合法帧，超时计数清零
                        c10b_active_flag=1;     // 标记数据有效
                        //拷贝：整体拷到 c10b_recv，业务层直接按字段取值
                        memcpy(&c10b_recv,c10b_buffer,c10b_datalen);
                    }
                }
                break;
        }
        Last_Recv = Usart_Receive;  // 记录本次字节供下次识别帧头
    }
}
```

6.9.3 K210 串口部分代码

```
import sensor , image , time , lcd
from fpioa_manager import fm  #串口引脚映射
from machine import UART  #从 machine 包导入 UART 类，K210 有 3 路硬件串口：
UART.UART1 / UART2 / UART3，本程序用的是 UART1

from board import board_info
from maix import GPIO
```

```
# 导入数据打包和解析类方法
import struct
from collections import OrderedDict # 用于创建有序字典
```

```
# 定时器模块
```

```
from machine import Timer
"""
将需要发送的数据打包 类
"""
class Uart_SendPack(): #发送端打包类
    def __init__(self,packmsg,dataformat):

        # 传输的数据包
        self.msg = packmsg
```

```
        # 传输的数据格式
        self.sendformat = dataformat
```

```
        # 要传输的数据长度
        self.data_len = struct.calcsize(self.sendformat)
```

```
    # BCC 校验函数
    def calculate_BCC(self,datalist,datalen):  #BCC 校验就是把指定范围内的
字节逐个做"异或(XOR)"，结果取低 8 位。
        ref = 0
        for i in range(datalen):
            ref = (ref^datalist[i])
        return ref&0xff
```

$$BCC = Head \oplus CamW_L \oplus CamW_H \oplus CamH_L \oplus CamH_H \oplus followX_L \oplus followX_H \oplus followY_L \oplus followY_H \oplus size_L \oplus size_H$$

```
    # 将要打包的数据进行 BCC 校验,并返回最终 BCC 校验的值
    def pack_BCC_Value(self):
        tmp_list = list( self.msg.values() ) # 将字典的值取出转换成列表
```

```
        tmp_packed = struct.pack(self.sendformat,*tmp_list) # 根据指定的数
据类型对列表值进行自动打包
```

```
        # 自动打包后的数值，进行 BCC 校验，获得最终需要发送的 BCC 校验值
        # 注意：这里只校验前 (总长-2) 个字节，也就是不含"校验位"和"帧尾"这两个，
        #       所以此刻 msg['BccCheck'] 里装的旧值是多少，都不影响计算结果。
        return self.calculate_BCC(tmp_packed,len(tmp_packed)-2)
```

```
    # 获取最终要发送的数据列表
    def get_Pack_List(self):
        tmplist = list(self.msg.values())# 将字典的值取出,转换成列表
        # struct.pack 返回的是 bytes 类型，这正是 uart.write() 需要的参数类
型，
        # 不需要再做 str() 或 encode() 转换。
        return struct.pack(self.sendformat,*tmplist)
```

```
""" 创建需要往 stm32 端发送数据的数据包 """
# 需要发送的数据(如果需要增删数据,则修改此处以及修改数据格式,同时在 32 端也对应增删即可)! ! !
#
# ============================================================================
# 串口通信协议（帧结构）—— 发送方向：K210 → STM32
# 这里用 OrderedDict 是为了"保证顺序"：字典的取值顺序必须和下面的格式串严格对应。
#
# ----------------------------------------------------------------------------
#  字节序号    字段        类型       说明
#   [0]      Head        uint8     帧头，固定 0xCC（STM32 靠它判断"一包开始了"）
#   [1][2]   Cam_W       uint16    图像宽 320，小端存储
#   [3][4]   Cam_H       uint16    图像高 240，小端存储
#   [5][6]   follow_x    uint16    目标中心 x 坐标（像素）
#   [7][8]   follow_y    uint16    目标中心 y 坐标（像素）
#   [9][10]  size        uint16    目标矩形面积 = 宽 × 高
#   [11]     BccCheck    uint8     前 11 个字节的异或校验值
#   [12]     End         uint8     帧尾，固定 0xDD
#                                        合计 13 字节
#
# ----------------------------------------------------------------------------
# 提醒：Cam_W / Cam_H 是写死的 320/240，如果改了摄像头分辨率，这里也要同步改。
#
# ============================================================================
send_pack1_msg = OrderedDict([
        ('Head',0xCC),    # 帧头           uint8_t 类型
        ('Cam_W', 320),   # 相机的像素宽度   uint16_t 类型
        ('Cam_H', 240),   # 相机的像素长度   uint16_t 类型
        ('follow_x',0),   # 需要跟踪的点 x    uint16_t 类型
        ('follow_y',0),   # 需要跟踪的点 y    uint16_t 类型
        ('size',0),       # 目标矩形面积     uint16_t 类型
        ('BccCheck',0),   # 数据 BCC 校验位    uint8_t 类型
        ('End',0xDD)      # 帧尾           uint8_t 类型
])
```

如果要自己增加传递的参数，可以在元组('BccCheck',0)前面添加。

```
# 数据格式 <代表小端模式，B 代表 uint8_t 类型,5H 代表 5 个 uint16_t 类型,2B 代表 2 个 uint8_t 类型
send_pack1_format = "<B5H2B"  #struct 模块的格式化字符串，用于把多个变量打包成二进制字节流
```

```
#实例化数据打包对象
send_pack1 = Uart_SendPack(send_pack1_msg,send_pack1_format)
```

```
# 更新需要发送的数据并返回打包结果,将结果直接发送到 stm32 端即可
def update_sendpack1_data(follow_x,follow_y,size):
    global send_pack1
    send_pack1.msg['follow_x'] = follow_x # 写入本次识别到的目标 x
    send_pack1.msg['follow_y'] = follow_y # 写入本次识别到的目标 y
    send_pack1.msg['size'] = size         # 写入本次识别到的目标面积
    # ★顺序很重要：必须"先把数据填好，再算 BCC"，否则算出来的校验值对不上。
    # 自己要添加的参数写在这里！！！
    send_pack1.msg['BccCheck'] = send_pack1.pack_BCC_Value()
    # 返回打包好的 13 字节 bytes，可直接交给 uart1.write()
    return send_pack1.get_Pack_List()
""" 创建需要往 stm32 端发送数据的数据包 END """
```

…

```
# 指定串口引脚并构造 UART 对象
#
================================================================================
======
# ★★★★ 串口初始化：整个串口功能最关键的三行 ★★★★
# K210 用串口分两步走：① 先用 FPIOA 声明引脚功能；② 再构造 UART 对象。两步缺一不可。
#
--------------------------------------------------------------------------------
------
# 引脚分配（可对照 STM32 端的接线）：
#   IO0 → UART1_RX ：K210 的"接收脚"，应接 STM32 的 TX
#   IO1 → UART1_TX ：K210 的"发送脚"，应接 STM32 的 RX
#   ★ 串口要交叉接线（TX→RX、RX→TX），并且两边必须共地(GND)！
#   ★ 若程序跑起来但 STM32 收不到数据，优先检查：这两个引脚是否被其它外设占用、
#     接线是否交叉、电平是否匹配。
#
--------------------------------------------------------------------------------
------
fm.register(0, fm.fpioa.UART1_RX)     # 把物理引脚 IO0 复用为 UART1 的接收功能
```

```
fm.register(1, fm.fpioa.UART1_TX)     # 把物理引脚 IO1 复用为 UART1 的发送功能
# 构造 UART 对象：
#   第 1 个参数 UART.UART1  —— 指定使用 1 号硬件串口（也可换成 UART2/UART3，但引脚要跟着改）
#   第 2 个参数 115200      —— 波特率，★必须与 STM32 端设置完全一致，否则收到全是乱码
# 其余参数（8 位数据位 / 无校验 / 1 位停止位）为默认值，两端用默认即可。
uart1 = UART(UART.UART1, 115200)
```

…

```
# 定时器中断回调函数
#   这个函数是"串口发送的实际触发者"：
#   定时器每 50ms 自动进来一次，把最新的识别结果打包并通过串口发出去。
#   也就是说串口发送频率 = 1000ms / 50ms = 20Hz（每秒发 20 帧）。
def timer_callback(timer):
    global g_follow_blob
    global g_last_blob
    global g_errortime

    # 最大最小面积定义(通过实测获得)
    min_area = 500
    max_area = 2500
```

```
    sendcx = 0
    sendcy = 0
    sendarea = 0
    try:
        # 将最大物体的中心坐标发送到 stm32
        if g_follow_blob!=None:
            # 计算识别到的面积
            sendarea = g_follow_blob.rect()[2]*g_follow_blob.rect()[3]
            sendcx = g_follow_blob.cx()
            sendcy = g_follow_blob.cy()
        #
=========================================================================
        # ★★★★ 真正执行发送的那一步 ★★★★
        # update_sendpack1_data() 先把 x / y / 面积填进数据包并算好 BCC，
        # 返回的是 13 字节的 bytes；uart1.write() 把这串字节原样从 IO1(TX) 推出去。
        # ★ uart1.write() 的参数必须是 bytes / bytearray，不能传字符串(str)。
        # ★ 这一句在"定时器中断"里执行，所以它不会阻塞主循环的图像处理。
        #
=========================================================================
```

```
        send_to_32 = update_sendpack1_data(sendcx,sendcy,sendarea)
        uart1.write(send_to_32)
```

```
    except Exception as e:
        print(e)
```

```
# 创建一个定时器,period 为定时时间,单位 ms.
# period=50 同时决定了"串口发送节拍": 每 50ms 触发一次 timer_callback。
timer = Timer(Timer.TIMER0, Timer.CHANNEL0, mode=Timer.MODE_PERIODIC,
period=50, callback=timer_callback)
```

```
# 启动定时器
timer.start()
```

6.9.4 大端（Big-Endian） & 小端（Little-Endian）

核心：多字节数据在内存 / 二进制流里，高低字节存放顺序。

只针对大于 1 字节的类型 (`H` uint16、`I` uint32)；单字节 `B` 不受大小端影响。

举个例子：16 位数 `0x1234`。

这个数字：高字节 = 0x12，低字节 = 0x34。

☑ 小端 Little-Endian（`<`）

低字节放前面，高字节放后面。

0x1234 → 内存 / 串口顺序：`0x34`，`0x12`。

低位在前，先发送低字节。

x86 PC、STM32、大部分嵌入式 MCU 默认小端。

☑ 大端 Big-Endian（`>`）

高字节放前面，低字节放后面

0x1234 → 内存 / 串口顺序：`0x12`，`0x34`。

高位在前，先发送高字节。

网络协议（TCP/IP）、部分 DSP、PowerPC 常用大端，也叫网络字节序。